Assessment and Answer Guide

DEVOPS ENGINEERING MASTERY — Volume 08, Companion Volume. First Edition, 9 November 2026. This guide contains 120 MCQs, 30 short-answer questions, 20 scenario-based troubleshooting questions, 10 incident case studies, the practical final examination and the capstone rubric — followed by a separate answer key with explanations. Attempt every question yourself before reading its answer; the explanation matters more than the letter.

> **WARNING:** Do not copy assignment solutions word for word. Read the outline, close this guide, and do the work yourself in your own Terraform project. That is where the learning happens.

মূল্যায়ন নির্দেশিকা • Assessment Guide

মূল্যায়ন নির্দেশিকা

এই নির্দেশিকা কীভাবে ব্যবহার করবেন

এই অধ্যায়টি বইয়ের মূল পাঠের সহযোগী। প্রথমে মূল অধ্যায়গুলো পড়ে ধারণা তৈরি করুন, তারপর নিজেকে যাচাই করতে এই মূল্যায়ন অংশটি ব্যবহার করুন। প্রশ্নের উত্তর দেওয়ার আগে বইয়ের সংশ্লিষ্ট অংশটি আবার পড়ে নিলে শেখা দৃঢ় হয়।

প্রতিটি প্রশ্নের পাশে একটি অসুবিধার স্তর দেওয়া আছে — **Beginner** মানে মৌলিক ধারণা, **Intermediate** মানে বাস্তব কাজে প্রয়োগ, আর **Advanced** মানে গভীর যুক্তি, ঝুঁকি ও ব্যতিক্রমী পরিস্থিতি। প্রথমবার Beginner প্রশ্নগুলো দিয়ে শুরু করুন; আত্মবিশ্বাস বাড়লে ধাপে ধাপে উপরের স্তরে যান।

বহুনির্বাচনি প্রশ্নের উত্তর প্রশ্নের পাশে দেওয়া হয়নি — উত্তর-নির্দেশিকা অংশে প্রতিটি উত্তরের ব্যাখ্যাসহ দেওয়া আছে। আগে নিজে উত্তর দেওয়ার চেষ্টা করুন, তারপর ব্যাখ্যা মিলিয়ে নিন। ভুল হলে শুধু সঠিক বিকল্পটি মুখস্থ না করে ব্যাখ্যাটি বুঝে নিন।

অসুবিধার স্তর

স্তর	অর্থ	যা যাচাই করে
Beginner	মৌলিক ধারণা	সংজ্ঞা, কমান্ডের কাজ, সাধারণ ব্যবহার
Intermediate	বাস্তব প্রয়োগ	কনফিগারেশন যুক্তি, state, backend, module, নিরাপত্তা
Advanced	গভীর বিচার	ব্যর্থতা, ঝুঁকি, migration, নীতি, জটিল নির্ভরতা

কীভাবে অনুশীলন করবেন

১. প্রতিটি অধ্যায় পড়ার পর সেই বিষয়ের MCQ গুলো দিন। ২. সংক্ষিপ্ত প্রশ্নগুলোর উত্তর নিজের ভাষায় লিখুন — মুখস্থ নয়, বোঝা যাচাই করাই লক্ষ্য। ৩. পরিস্থিতি (scenario) ও কেস স্টাডিগুলো বন্ধুর সাথে আলোচনা করে সমাধান করুন; বাস্তব কাজে এগুলোই সবচেয়ে বেশি কাজে লাগে। ৪. শেষে চূড়ান্ত পরীক্ষার কাজগুলো হাতে-কলমে একটি test পরিবেশে করে দেখুন।

সতর্কতা — নকল নয়, নিজের বোঝা লিখুন: অ্যাসাইনমেন্ট বা পরীক্ষার উত্তরে এই বইয়ের সমাধান হুবহু কপি করবেন না। এখানকার উত্তরগুলো শেখার জন্য — ধারণাটি বুঝে নিজের ভাষায়, নিজের উদাহরণ দিয়ে লিখুন। হুবহু কপি করলে শেখা হয় না, আর মূল্যায়নে তা ধরা পড়ে।

মনে রাখবেন: Terraform শেখার আসল পরীক্ষা হলো plan পড়ে বুঝতে পারা — কী তৈরি হবে, কী বদলাবে, কী ধ্বংস হবে। প্রতিটি অনুশীলনে plan-এর আউটপুট মন দিয়ে পড়ার অভ্যাস করুন।

বহুনির্বাচনি প্রশ্ন • Multiple Choice Questions

বহুনির্বাচনি প্রশ্ন

বহুনির্বাচনি প্রশ্ন

মোট ১৩৮টি প্রশ্ন। প্রতিটি প্রশ্নে চারটি বিকল্প আছে — শুধু একটি সঠিক। উত্তর দেওয়ার পর উত্তর-নির্দেশিকা অংশে ব্যাখ্যাসহ মিলিয়ে নিন।

১. Infrastructure as Code (IaC) বলতে মূলত কী বোঝায়? [Beginner]

ক) অবকাঠামো শুধু কনসোলে হাতে তৈরি করা
খ) অবকাঠামোকে সংস্করণ-নিয়ন্ত্রিত কোড হিসেবে সংজ্ঞায়িত ও পরিচালনা করা
গ) শুধু সার্ভারে স্ক্রিপ্ট চালানো
ঘ) ক্লাউড বিল কমানোর একটি টুল

২. Declarative পদ্ধতির মূল বৈশিষ্ট্য কোনটি? [Beginner]

ক) ধাপে ধাপে কীভাবে করতে হবে তা লেখা
খ) শেষ অবস্থা (desired state) ঘোষণা করা; টুল কীভাবে পৌঁছাবে তা ঠিক করে
গ) প্রতিবার ম্যানুয়াল অনুমোদন নেওয়া
ঘ) শুধু একটি ক্লাউডে কাজ করা

৩. Imperative পদ্ধতির উদাহরণ কোনটি? [Beginner]

ক) Terraform configuration লেখা
খ) Ansible playbook-এ ধাপে ধাপে কমান্ড দেওয়া নয় — আসলে Bash স্ক্রিপ্টে create → configure ধাপ লেখা
গ) Kubernetes manifest লেখা
ঘ) CloudFormation template লেখা

৪. Terraform-এর প্রধান কাজ কী? [Beginner]

ক) অ্যাপ্লিকেশন কোড কম্পাইল করা
খ) বিভিন্ন ক্লাউডে অবকাঠামো provision ও পরিচালনা করা
গ) ডাটাবেসের ভেতরের ডেটা মাইগ্রেট করা
ঘ) নেটওয়ার্ক প্যাকেট বিশ্লেষণ করা

৫. HCL-এর পূর্ণরূপ কী? [Beginner]

ক) HashiCorp Configuration Language
খ) High Cloud Language
গ) Hybrid Compute Language
ঘ) Host Configuration Library

৬. Terraform-এ একটি resource ব্লক কী প্রকাশ করে? [Beginner]

ক) একটি ডেটা সোর্স থেকে পড়া তথ্য
খ) পরিচালনা করতে হবে এমন একটি অবকাঠামো উপাদান

গ) একটি লোকাল ভেরিয়েবল
ঘ) একটি আউটপুট মান

৭. data ব্লকের কাজ কী? [Beginner]

ক) নতুন রিসোর্স তৈরি করা
খ) বিদ্যমান অবকাঠামো বা বাইরের তথ্য পড়ে ব্যবহার করা
গ) state ফাইল মুছে ফেলা
ঘ) provider ইনস্টল করা

৮. terraform init কী করে? [Beginner]

ক) রিসোর্স তৈরি করে
খ) provider plugin ডাউনলোড করে ও backend কনফিগার করে
গ) state ফাইল মুছে দেয়
ঘ) plan দেখায়

৯. terraform plan-এর উদ্দেশ্য কী? [Beginner]

ক) রিসোর্স ধ্বংস করা
খ) পরিবর্তন প্রয়োগ না করে কী বদলাবে তা দেখানো
গ) state লক খোলা
ঘ) মডিউল প্রকাশ করা

১০. terraform apply কী করে? [Beginner]

ক) শুধু plan ফাইল পড়ে
খ) পরিকল্পিত পরিবর্তন বাস্তবে প্রয়োগ করে
গ) কোড ফরম্যাট করে
ঘ) ভেরিয়েবল মুছে দেয়

১১. Terraform state ফাইলে কী থাকে? [Beginner]

ক) শুধু provider-এর নাম
খ) বর্তমান অবকাঠামোর অবস্থা ও রিসোর্স-কনফিগারেশন mapping
গ) অ্যাপ্লিকেশনের সোর্স কোড
ঘ) CI পাইপলাইনের লগ

১২. state ফাইল হারিয়ে গেলে কী হয়? [Intermediate]

ক) কিছুই হয় না
খ) Terraform বিদ্যমান রিসোর্স চিনতে পারে না; পুনরায় তৈরি করতে চাইবে
গ) provider মুছে যায়
ঘ) কোড মুছে যায়

১৩. Remote backend ব্যবহারের প্রধান সুবিধা কী? [Beginner]

ক) plan দ্রুত হয়
খ) দলগতভাবে একই state শেয়ার ও লক করা যায়

গ) খরচ কমে
ঘ) কোড ছোট হয়

১৪. State locking কেন দরকার? [Intermediate]

ক) plan দ্রুত করতে
খ) একই সময়ে দুটি apply যাতে state নষ্ট না করে
গ) বিল কমাতে
ঘ) provider আপডেট করতে

১৫. terraform destroy কী করে? [Beginner]

ক) শুধু একটি রিসোর্স মুছে দেয়
খ) কনফিগারেশনের সব পরিচালিত রিসোর্স ধ্বংস করে
গ) state ব্যাকআপ নেয়
ঘ) workspace বদলায়

১৬. variable ব্লকের কাজ কী? [Beginner]

ক) মান হার্ডকোড করা
খ) কনফিগারেশনকে পুনর্ব্যবহারযোগ্য করতে ইনপুট মান নেওয়া
গ) আউটপুট দেখানো
ঘ) state লক করা

১৭. output ব্লক কেন লেখা হয়? [Beginner]

ক) রিসোর্স মুছতে
খ) তৈরি রিসোর্সের গুরুত্বপূর্ণ মান (যেমন IP, ARN) প্রকাশ করতে
গ) provider বদলাতে
ঘ) state এনক্রিপ্ট করতে

১৮. locals ব্লকের সুবিধা কী? [Beginner]

ক) বাইরে থেকে মান নেওয়া
খ) একই expression বারবার না লিখে স্থানীয়ভাবে নাম দিয়ে ব্যবহার করা
গ) state শেয়ার করা
ঘ) provider লক করা

১৯. string, number, bool কোন ধরনের টাইপ? [Beginner]

ক) collection type
খ) primitive type
গ) structural type
ঘ) dynamic type

২০. list(string) ও set(string)-এর পার্থক্য কী? [Intermediate]

ক) কোনো পার্থক্য নেই
খ) list-এ ক্রম ও ডুপ্লিকেট থাকে, set-এ ক্রম নেই ও ডুপ্লিকেট নেই

গ) set শুধু সংখ্যা নেয়
ঘ) list শুধু map নেয়

২১. map(string) কী প্রকাশ করে? [Beginner]

ক) সংখ্যার তালিকা
খ) key-value জোড়ার সংগ্রহ
গ) একটি সংখ্যা
ঘ) একটি বুলিয়ান

২২. object টাইপ কখন ব্যবহার করা হয়? [Intermediate]

ক) যখন বিভিন্ন attribute-এর নির্দিষ্ট কাঠামো দরকার
খ) যখন শুধু সংখ্যা দরকার
গ) যখন লুপ দরকার
ঘ) যখন state লক দরকার

২৩. variable-এর default না থাকলে কী হয়? [Beginner]

ক) Terraform নিজে মান বানায়
খ) apply-এর সময় মান দিতে হয়, না দিলে ত্রুটি হয়
গ) variable উপেক্ষা করা হয়
ঘ) plan চলে কিন্তু apply ব্যর্থ হয়

২৪. sensitive = true-এর কাজ কী? [Intermediate]

ক) মান এনক্রিপ্ট করে state-এ লুকায়
খ) CLI আউটপুটে মান দেখানো বন্ধ করে (তবে state-এ থাকে)
গ) মান মুছে দেয়
ঘ) provider বদলায়

২৫. depends_on কখন ব্যবহার করা হয়? [Intermediate]

ক) সবসময়
খ) যখন implicit dependency যথেষ্ট নয়, তখন স্পষ্ট নির্ভরতা বোঝাতে
গ) শুধু module-এ
ঘ) শুধু data source-এ

২৬. Implicit dependency কীভাবে তৈরি হয়? [Beginner]

ক) depends_on লিখলে
খ) একটি রিসোর্সের attribute অন্য রিসোর্সে ব্যবহার করলে
গ) count ব্যবহার করলে
ঘ) workspace বদলালে

২৭. count = 3 দিলে কী হয়? [Beginner]

ক) একটি রিসোর্স তিনবার plan হয়
খ) একই রিসোর্সের তিনটি instance তৈরি হয়

গ) তিনটি workspace তৈরি হয়
ঘ) তিনটি provider লাগে

২৮. count.index-এর মান কী হয়? [Beginner]

ক) সবসময় ১
খ) ০ থেকে শুরু হওয়া ক্রমিক সংখ্যা
গ) রিসোর্সের নাম
ঘ) provider-এর সংস্করণ

২৯. for_each কখন count-এর চেয়ে ভালো? [Intermediate]

ক) যখন সংখ্যা নির্দিষ্ট
খ) যখন প্রতিটি instance-কে স্থিতিশীল key (যেমন নাম) দিয়ে চেনা দরকার
গ) যখন একটি instance দরকার
ঘ) যখন লুপ দরকার নেই

৩০. for_each-এ map-এর key বদলালে কী হয়? [Intermediate]

ক) কিছুই হয় না
খ) পুরোনো key-এর instance ধ্বংস হয়ে নতুন key-এর instance তৈরি হয়
গ) শুধু নাম বদলায়
ঘ) state লক হয়

৩১. each.key ও each.value কী? [Beginner]

ক) count-এর index
খ) for_each-এর বর্তমান instance-এর key ও মান
গ) provider-এর তথ্য
ঘ) workspace-এর নাম

৩২. lifecycle { prevent_destroy = true } কী করে? [Intermediate]

ক) রিসোর্স দ্রুত ধ্বংস করে
খ) ধ্বংস করার plan-কে ত্রুটি দিয়ে আটকায়
গ) রিসোর্স আপডেট বন্ধ করে
ঘ) state মুছে দেয়

৩৩. lifecycle { create_before_destroy = true } কখন দরকার? [Intermediate]

ক) যখন downtime এড়িয়ে আগে নতুনটি তৈরি করে পরে পুরোনোটি মুছতে হয়
খ) যখন রিসোর্স কখনো বদলাবে না
গ) যখন খরচ কমাতে হয়
ঘ) যখন state লক করতে হয়

৩৪. lifecycle { ignore_changes } কী করে? [Intermediate]

ক) সব পরিবর্তন উপেক্ষা করে
খ) নির্দিষ্ট attribute-এর বাইরের পরিবর্তন plan-এ দেখায় না

গ) রিসোর্স মুছে দেয়
ঘ) provider উপেক্ষা করে

৩৫. terraform taint (বর্তমানে replace) কী করে? [Intermediate]

ক) রিসোর্স মুছে দেয়
খ) পরের apply-তে রিসোর্সটি ধ্বংস করে পুনরায় তৈরি করতে চিহ্নিত করে
গ) state লক খোলে
ঘ) plan মুছে দেয়

৩৬. Dynamic block কখন ব্যবহার করা হয়? [Intermediate]

ক) যখন nested block-এর সংখ্যা variable-এর ওপর নির্ভর করে
খ) যখন একটি মাত্র block দরকার
গ) যখন provider বদলাতে হয়
ঘ) যখন state মুছতে হয়

৩৭. Terraform function-এর উদাহরণ কোনটি? [Beginner]

ক) resource
খ) merge(), length(), format()
গ) provider
ঘ) backend

৩৮. format("%s-%s", "web", "01") কী দেবে? [Beginner]

ক) web01
খ) web-01
গ) web_01
ঘ) 01-web

৩৯. length(["a", "b"]) কত? [Beginner]

ক) ১
খ) ২
গ) ৩
ঘ) ০

৪০. Module কী? [Beginner]

ক) একটি provider
খ) পুনর্ব্যবহারযোগ্য Terraform কনফিগারেশনের সংগ্রহ
গ) একটি state ফাইল
ঘ) একটি workspace

৪১. module ব্লকে source কী নির্দেশ করে? [Beginner]

ক) provider-এর নাম
খ) মডিউলের অবস্থান — local path, Git বা Registry

গ) state-এর অবস্থান
ঘ) workspace-এর নাম

৪২. Module version pin করা কেন জরুরি? [Intermediate]

ক) খরচ কমে
খ) অপ্রত্যাশিত breaking change থেকে রক্ষা পেতে নির্দিষ্ট সংস্করণ ব্যবহার করা যায়
গ) plan দ্রুত হয়
ঘ) state ছোট হয়

৪৩. Terraform Registry কী? [Beginner]

ক) একটি ক্লাউড
খ) provider ও module খোঁজা ও ব্যবহারের সরকারি ভান্ডার
গ) একটি state backend
ঘ) একটি CI টুল

৪৪. Child module-এর output parent কীভাবে পায়? [Intermediate]

ক) সরাসরি state পড়ে
খ) module.<নাম>.<output> দিয়ে
গ) variable দিয়ে
ঘ) data source দিয়ে

৪৫. terraform import কী করে? [Intermediate]

ক) নতুন রিসোর্স তৈরি করে
খ) বিদ্যমান বাস্তব রিসোর্সকে state-এ নিয়ে আসে
গ) কোড আমদানি করে
ঘ) provider আপডেট করে

৪৬. Import করার পর প্রথম plan-এ পার্থক্য দেখালে কী বোঝায়? [Intermediate]

ক) import ব্যর্থ হয়েছে
খ) কনফিগারেশন বাস্তব অবস্থার সাথে মেলেনি; কনফিগারেশন মেলাতে হবে
গ) state নষ্ট হয়েছে
ঘ) provider ভুল

৪৭. Drift কী? [Intermediate]

ক) state ফাইলের আকার বাড়া
খ) কনফিগারেশনের বাইরে বাস্তব অবকাঠামোতে পরিবর্তন
গ) provider আপডেট
ঘ) workspace বদল

৪৮. terraform refresh (বর্তমানে plan -refresh-only) কী করে? [Intermediate]

ক) কনফিগারেশন বদলায়
খ) বাস্তব অবস্থা পড়ে state আপডেট করে

গ) রিসোর্স তৈরি করে
ঘ) state মুছে দেয়

৪৯. Workspace কী? [Intermediate]

ক) একটি আলাদা provider
খ) একই কনফিগারেশন দিয়ে একাধিক environment-এর আলাদা state
গ) একটি মডিউল
ঘ) একটি backend

৫০. terraform workspace new prod কী করে? [Beginner]

ক) prod নামে রিসোর্স তৈরি করে
খ) prod নামে নতুন workspace ও খালি state তৈরি করে
গ) backend বদলায়
ঘ) provider বদলায়

৫১. Provider version constraint ~> 5.0-এর অর্থ কী? [Intermediate]

ক) শুধু 5.0
খ) 5.x সংস্করণ (5.0 থেকে 6.0-এর আগে)
গ) যেকোনো সংস্করণ
ঘ) 4.x সংস্করণ

৫২. .terraform.lock.hcl ফাইলের কাজ কী? [Intermediate]

ক) state সংরক্ষণ
খ) provider-এর নির্দিষ্ট সংস্করণ ও hash লক করা
গ) workspace তালিকা
ঘ) variable-এর মান

৫৩. terraform fmt কী করে? [Beginner]

ক) plan চালায়
খ) কোডের বিন্যাস একরকম করে
গ) state যাচাই করে
ঘ) provider ডাউনলোড করে

৫৪. terraform validate কী যাচাই করে? [Beginner]

ক) ক্লাউডে রিসোর্স আছে কি না
খ) কনফিগারেশনের syntax ও reference ঠিক আছে কি না
গ) খরচ কত
ঘ) state লক আছে কি না

৫৫. S3 backend-এ state locking কী দিয়ে হয়? [Intermediate]

ক) S3 versioning
খ) DynamoDB টেবিল

গ) IAM role
ঘ) CloudWatch

৫৬. S3 backend-এ state-এর নিরাপত্তায় কী জরুরি? [Intermediate]

ক) public করা
খ) versioning ও encryption চালু করা
গ) bucket নাম ছোট রাখা
ঘ) lifecycle বন্ধ করা

৫৭. azurerm backend কোথায় state রাখে? [Intermediate]

ক) S3-তে
খ) Azure Storage Account-এর Blob-এ
গ) GCS-এ
ঘ) লোকাল ডিস্কে

৫৮. GCS backend কোথায় state রাখে? [Beginner]

ক) S3-তে
খ) Google Cloud Storage bucket-এ
গ) Azure Blob-এ
ঘ) DynamoDB-তে

৫৯. Secret (যেমন পাসওয়ার্ড) Terraform-এ কীভাবে রাখা উচিত নয়? [Beginner]

ক) Git-এ plain text-এ
খ) Vault বা Secrets Manager থেকে data source দিয়ে পড়ে
গ) environment variable দিয়ে
ঘ) CI secret দিয়ে

৬০. state ফাইলে sensitive মান থাকলে কী ঝুঁকি? [Intermediate]

ক) কোনো ঝুঁকি নেই
খ) state পড়তে পারলে secret ফাঁস হয়
গ) plan ধীর হয়
ঘ) provider বদলে যায়

৬১. Policy as Code বলতে কী বোঝায়? [Intermediate]

ক) নীতিমালা কাগজে লেখা
খ) নিয়ম (যেমন encryption বাধ্যতামূলক) কোডে লিখে স্বয়ংক্রিয়ভাবে যাচাই করা
গ) provider-এর নীতি
ঘ) বিলের নীতি

৬২. OPA-র কাজ কী? [Advanced]

ক) provider ইনস্টল করা
খ) Rego নীতিমালা দিয়ে plan যাচাই করা

গ) state ব্যাকআপ নেওয়া
ঘ) খরচ হিসাব করা

৬৩. Checkov কী যাচাই করে? [Intermediate]

ক) কোডের বানান
খ) Terraform কনফিগারেশনে নিরাপত্তা ও compliance ত্রুটি
গ) নেটওয়ার্ক গতি
ঘ) বিল

৬৪. TFLint কী করে? [Intermediate]

ক) state মুছে দেয়
খ) Terraform কোডে ভুল, deprecated syntax ও provider নিয়ম যাচাই করে
গ) apply চালায়
ঘ) secret তৈরি করে

৬৫. CI-তে terraform plan কেন চালানো হয়? [Beginner]

ক) apply করতে
খ) pull request-এ পরিবর্তন পর্যালোচনা করতে
গ) state মুছতে
ঘ) খরচ বাড়াতে

৬৬. terraform plan -out=tfplan কী করে? [Intermediate]

ক) plan মুছে দেয়
খ) plan ফাইল সংরক্ষণ করে পরে একই plan apply করা যায়
গ) state লক খোলে
ঘ) খরচ দেখায়

৬৭. apply-এর আগে plan পর্যালোচনা কেন জরুরি? [Beginner]

ক) খরচ বাড়াতে
খ) অপ্রত্যাশিত destroy বা replacement ধরা পড়ে
গ) provider বদলাতে
ঘ) workspace বদলাতে

৬৮. AWS-এ Terraform সাধারণত কোন provider ব্যবহার করে? [Beginner]

ক) azurerm
খ) hashicorp/aws
গ) google
ঘ) random

৬৯. aws_instance রিসোর্সে ami ও instance_type কী? [Beginner]

ক) output
খ) প্রয়োজনীয় argument — কোন image ও কোন আকারের সার্ভার

গ) variable
ঘ) data source

৭০. Security group-এ 0.0.0.0/0 দিয়ে SSH খোলা কেন ঝুঁকিপূর্ণ? [Beginner]

ক) খরচ বাড়ে
খ) ইন্টারনেটের যে কেউ SSH চেষ্টা করতে পারে
গ) গতি কমে
ঘ) state বড় হয়

৭১. S3 bucket-এ public access বন্ধ রাখা কেন জরুরি? [Beginner]

ক) খরচ কমে
খ) অননুমোদিত ব্যক্তি ডেটা পড়তে বা বদলাতে পারে
গ) গতি বাড়ে
ঘ) state ছোট হয়

৭২. Azure-এ resource group কী? [Beginner]

ক) একটি VM
খ) রিসোর্স সংগঠিত করার ধারক (container)
গ) একটি নেটওয়ার্ক
ঘ) একটি secret

৭৩. GCP-এ Terraform provider-এর নাম কী? [Beginner]

ক) hashicorp/azurerm
খ) hashicorp/google
গ) hashicorp/aws
ঘ) hashicorp/random

৭৪. IBM Cloud-এ Terraform provider কী দিয়ে চেনা যায়? [Intermediate]

ক) hashicorp/aws
খ) IBM-Cloud/ibm
গ) hashicorp/google
ঘ) hashicorp/azurerm

৭৫. terraform show কী দেখায়? [Beginner]

ক) কোড
খ) বর্তমান state বা plan ফাইলের মানব-পাঠযোগ্য বিবরণ
গ) provider তালিকা
ঘ) workspace তালিকা

৭৬. terraform output কী করে? [Beginner]

ক) output মুছে দেয়
খ) ঘোষিত output-এর মান দেখায়

গ) variable বদলায়
ঘ) state লক করে

৭৭. terraform state list কী দেখায়? [Intermediate]

ক) provider তালিকা
খ) state-এর সব রিসোর্সের ঠিকানা
গ) workspace তালিকা
ঘ) খরচ

৭৮. terraform state mv কী করে? [Advanced]

ক) রিসোর্স মুছে দেয়
খ) state-এ রিসোর্সের ঠিকানা বদলায় (যেমন module-এ সরানো)
গ) রিসোর্স তৈরি করে
ঘ) backend বদলায়

৭৯. terraform state rm কী করে? [Advanced]

ক) রিসোর্স মুছে দেয়
খ) state থেকে রিসোর্স সরায়, বাস্তব রিসোর্স থেকে যায়
গ) backend মুছে দেয়
ঘ) workspace মুছে দেয়

৮০. terraform graph কী তৈরি করে? [Intermediate]

ক) plan ফাইল
খ) রিসোর্স dependency-এর গ্রাফ (DOT ফরম্যাট)
গ) state ব্যাকআপ
ঘ) খরচের হিসাব

৮১. terraform providers কমান্ড কী দেখায়? [Beginner]

ক) খরচ
খ) কনফিগারেশনে ব্যবহৃত provider ও তাদের সংস্করণ
গ) state-এর আকার
ঘ) workspace

৮২. terraform version কী দেখায়? [Beginner]

ক) state-এর মান
খ) Terraform ও provider-এর সংস্করণ
গ) খরচ
ঘ) workspace তালিকা

৮৩. Provider upgrade-এ breaking change এলে প্রথমে কী করা উচিত? [Intermediate]

ক) সরাসরি production-এ apply করা
খ) changelog পড়ে non-production-এ plan যাচাই করা

গ) state মুছে দেওয়া
ঘ) lock ফাইল মুছে দেওয়া

৮৪. required_providers ব্লক কী নির্ধারণ করে? [Intermediate]

ক) backend
খ) কোন provider, কোন source ও কোন সংস্করণ অনুমোদিত
গ) workspace
ঘ) variable-এর মান

৮৫. required_version কী করে? [Beginner]

ক) provider বদলায়
খ) Terraform CLI-এর ন্যূনতম/অনুমোদিত সংস্করণ নির্ধারণ করে
গ) state লক করে
ঘ) workspace তৈরি করে

৮৬. Backend বদলালে কী করতে হয়? [Advanced]

ক) কিছুই না
খ) terraform init -migrate-state দিয়ে state স্থানান্তর করা
গ) apply করা
ঘ) state মুছে দেওয়া

৮৭. terraform init -upgrade কী করে? [Intermediate]

ক) state মুছে দেয়
খ) provider ও module-কে অনুমোদিত নতুন সংস্করণে আপডেট করে
গ) workspace মুছে দেয়
ঘ) plan চালায়

৮৮. Count দিয়ে তৈরি instance-এর মাঝের একটি মুছলে কী হয়? [Advanced]

ক) সব instance পুনরায় তৈরি হয়
খ) পরের instance-গুলোর index সরে গিয়ে সেগুলো replace হতে পারে
গ) কিছুই হয় না
ঘ) state নষ্ট হয়

৮৯. for_each-এ set ব্যবহার করলে each.key কী হয়? [Intermediate]

ক) index সংখ্যা
খ) set-এর প্রতিটি মান
গ) provider নাম
ঘ) workspace নাম

৯০. Module-এর ভেতরে provider কনফিগার করা উচিত কি? [Advanced]

ক) হ্যাঁ, সবসময়
খ) সাধারণত না — root module provider দেয়, child module তা উত্তরাধিকার পায়

গ) শুধু AWS-এ
ঘ) শুধু local module-এ

৯১. Module-এ variable validation কী করে? [Intermediate]

ক) খরচ কমায়
খ) ভুল মান (যেমন খালি নাম) আগেই ত্রুটি দিয়ে আটকায়
গ) state লক করে
ঘ) plan দ্রুত করে

৯২. terraform test (test framework) কী করে? [Advanced]

ক) apply চালায়
খ) .tftest.hcl ফাইলে লেখা assertion দিয়ে কনফিগারেশন যাচাই করে
গ) state মুছে দেয়
ঘ) provider ডাউনলোড করে

৯৩. Terratest কী? [Advanced]

ক) একটি provider
খ) Go-ভিত্তিক Terraform টেস্ট ফ্রেমওয়ার্ক যা আসল অবকাঠামো তৈরি করে যাচাই করে
গ) একটি backend
ঘ) একটি registry

৯৪. Cost estimation-এ Infracost কী করে? [Intermediate]

ক) বিল পরিশোধ করে
খ) plan থেকে আনুমানিক মাসিক খরচ দেখায়
গ) রিসোর্স তৈরি করে
ঘ) state এনক্রিপ্ট করে

৯৫. count ভুল করে 100 দিলে কী ঝুঁকি? [Intermediate]

ক) কিছুই না
খ) অপ্রত্যাশিতভাবে 100টি রিসোর্স তৈরি হয়ে বিল বিস্ফোরণ হতে পারে
গ) state ছোট হয়
ঘ) plan দ্রুত হয়

৯৬. plan-এ অপ্রত্যাশিত destroy দেখলে প্রথমে কী করা উচিত? [Intermediate]

ক) সাথে সাথে apply করা
খ) apply থামিয়ে কারণ খোঁজা — নাম, key বা কনফিগারেশন বদলেছে কি না
গ) state মুছে দেওয়া
ঘ) workspace বদলানো

৯৭. State lock আটকে গেলে প্রথমে কী যাচাই করা উচিত? [Intermediate]

ক) সাথে সাথে force-unlock করা
খ) আর কোনো apply চলছে কি না; চললে অপেক্ষা করা

গ) state মুছে দেওয়া
ঘ) provider বদলানো

৯৮. terraform force-unlock কখন ব্যবহার করা হয়? [Advanced]

ক) সবসময়
খ) যখন নিশ্চিত যে কোনো apply চলছে না, কিন্তু lock আটকে আছে
গ) প্রতিটি apply-এর আগে
ঘ) plan-এর সময়

৯৯. Sensitive output state-এ কীভাবে সুরক্ষিত রাখা যায়? [Intermediate]

ক) Git-এ রাখলে
খ) encrypted remote backend + কঠোর IAM + state-এর access সীমিত করে
গ) output মুছে দিলে
ঘ) plan না চালালে

১০০. Backend bucket মুছে গেলে কী হয়? [Advanced]

ক) কিছুই হয় না
খ) state হারিয়ে যায়; ব্যাকআপ/versioning না থাকলে পুনরুদ্ধার কঠিন হয়
গ) provider মুছে যায়
ঘ) কোড মুছে যায়

১০১. Git-এ .tfstate ফাইল কমিট করা উচিত কি? [Beginner]

ক) হ্যাঁ, সবসময়
খ) না — state-এ secret থাকতে পারে ও দলগত lock হয় না
গ) শুধু ছোট প্রজেক্টে
ঘ) শুধু public repo-তে

১০২. .gitignore-এ কী রাখা উচিত? [Beginner]

ক) *.tf
খ) .terraform/, *.tfstate, *.tfplan, crash.log
গ) README.md
ঘ) *.md

১০৩. terraform workspace select dev কী করে? [Beginner]

ক) dev রিসোর্স তৈরি করে
খ) বর্তমান workspace dev-এ বদলায়
গ) dev backend তৈরি করে
ঘ) dev provider ইনস্টল করে

১০৪. একই কোডে dev ও prod আলাদা করতে workspace-এর বিকল্প কী? [Intermediate]

ক) একই state ব্যবহার করা
খ) আলাদা ডিরেক্টরি/আলাদা backend key বা tfvars ফাইল ব্যবহার করা

গ) provider বদলানো
ঘ) state মুছে দেওয়া

১০৫. tfvars ফাইলের কাজ কী? [Beginner]

ক) provider ইনস্টল করা
খ) variable-এর মান ফাইলে দিয়ে পরিবেশভেদে ব্যবহার করা
গ) state লক করা
ঘ) module প্রকাশ করা

১০৬. Environment variable TF_VAR_ কী করে? [Intermediate]

ক) provider বদলায়
খ) TF_VAR_<নাম> দিয়ে variable-এর মান দেয়
গ) state মুছে দেয়
ঘ) workspace তৈরি করে

১০৭. terraform plan -target=aws_instance.web কী করে? [Advanced]

ক) সব রিসোর্স apply করে
খ) শুধু নির্দিষ্ট রিসোর্স ও তার dependency নিয়ে plan করে
গ) রিসোর্স মুছে দেয়
ঘ) state লক খোলে

১০৮. depends_on অতিরিক্ত ব্যবহারের সমস্যা কী? [Advanced]

ক) plan দ্রুত হয়
খ) অপ্রয়োজনীয় ক্রম তৈরি হয়ে parallel কাজ কমে ও graph জটিল হয়
গ) খরচ কমে
ঘ) state ছোট হয়

১০৯. Resource replacement কখন ঘটে? [Intermediate]

ক) attribute বদলালে সবসময়
খ) যখন এমন argument বদলায় যা ForceNew — যেমন নাম বা AMI
গ) output বদলালে
ঘ) variable বদলালে

১১০. Data source-এর মান কখন পড়া হয়? [Intermediate]

ক) apply-এর শেষে
খ) plan-এর সময় (বেশিরভাগ ক্ষেত্রে)
গ) destroy-এর সময়
ঘ) init-এর সময়

১১১. random_password resource কেন sensitive হিসেবে ধরা হয়? [Intermediate]

ক) এটি provider
খ) এর মান state-এ থাকে ও আউটপুটে গোপন রাখা উচিত

গ) এটি module
ঘ) এটি data source

১১২. null_resource কী কাজে লাগে? [Advanced]

ক) রিসোর্স তৈরি করতে
খ) provisioner বা ট্রিগার-ভিত্তিক কাজ চালাতে (বর্তমানে terraform_data দিয়ে প্রতিস্থাপিত)
গ) state লক করতে
ঘ) provider বদলাতে

১১৩. Provisioner ব্যবহারের ব্যাপারে HashiCorp-এর পরামর্শ কী? [Advanced]

ক) সবসময় ব্যবহার করা
খ) শেষ উপায় হিসেবে ব্যবহার করা; তার আগে provider/cloud-init ব্যবহার করা
গ) কখনো ব্যবহার না করা
ঘ) শুধু destroy-এ ব্যবহার করা

১১৪. local-exec provisioner কী করে? [Advanced]

ক) দূরের সার্ভারে চলে
খ) Terraform চালানো মেশিনে কমান্ড চালায়
গ) state লক করে
ঘ) provider ইনস্টল করে

১১৫. remote-exec provisioner কী করে? [Advanced]

ক) লোকাল মেশিনে চলে
খ) তৈরি রিসোর্সে SSH/WinRM দিয়ে কমান্ড চালায়
গ) state মুছে দেয়
ঘ) backend বদলায়

১১৬. Module registry-তে private module কীভাবে ব্যবহার করা হয়? [Advanced]

ক) public registry-তে
খ) Terraform Cloud/Enterprise private registry বা Git source দিয়ে
গ) state-এ
ঘ) workspace-এ

১১৭. Git source-এ module version কীভাবে নির্দিষ্ট করা হয়? [Advanced]

ক) version argument দিয়ে
খ) URL-এ ?ref=<tag/branch> দিয়ে
গ) lock ফাইলে
ঘ) workspace-এ

১১৮. terraform init -backend=false কখন ব্যবহার করা হয়? [Advanced]

ক) apply-এর সময়
খ) backend কনফিগার না করে শুধু module/provider প্রস্তুত করতে

গ) state মুছতে
ঘ) plan-এ

১১৯. plan-এ "No changes" দেখালে কী বোঝায়? [Beginner]

ক) ত্রুটি হয়েছে
খ) বাস্তব অবস্থা কনফিগারেশনের সাথে মিলে গেছে
গ) state মুছে গেছে
ঘ) provider নেই

১২০. Apply ব্যর্থ হয়ে মাঝপথে থামলে কী করতে হয়? [Intermediate]

ক) state মুছে দেওয়া
খ) ত্রুটি ঠিক করে আবার plan/apply করা; state-এ আংশিক অবস্থা থাকে
গ) workspace মুছে দেওয়া
ঘ) provider বদলানো

১২১. S3 backend-এ DynamoDB টেবিলের নাম ভুল হলে কী হয়? [Intermediate]

ক) plan দ্রুত হয়
খ) lock নিতে ব্যর্থ হয়ে init/apply ত্রুটি দেয়
গ) state মুছে যায়
ঘ) খরচ কমে

১২২. Azure backend-এ state lock কীভাবে কাজ করে? [Intermediate]

ক) DynamoDB দিয়ে
খ) Blob lease দিয়ে
গ) S3 দিয়ে
ঘ) GCS দিয়ে

১২৩. GCP backend-এ state lock কীভাবে হয়? [Advanced]

ক) DynamoDB দিয়ে
খ) GCS object-এর মাধ্যমে (native locking)
গ) Blob lease দিয়ে
ঘ) S3 দিয়ে

১২৪. terraform destroy -target ব্যবহারের ঝুঁকি কী? [Advanced]

ক) কিছুই না
খ) শুধু একটি রিসোর্স মুছে dependency ভেঙে যেতে পারে
গ) state বড় হয়
ঘ) খরচ কমে

১২৫. Variable type না দিলে কী হয়? [Beginner]

ক) ত্রুটি হয়
খ) Terraform মান থেকে type অনুমান করে (dynamic)

গ) variable উপেক্ষা হয়
ঘ) plan থেমে যায়

১২৬. cidrsubnet() function কী করে? [Advanced]

ক) IP মুছে দেয়
খ) CIDR ব্লক থেকে সাবনেট বের করে
গ) DNS তৈরি করে
ঘ) security group খোলে

১২৭. toset() কেন ব্যবহার করা হয়? [Intermediate]

ক) তালিকাকে set-এ রূপান্তর করে for_each-এ ব্যবহার করতে
খ) string বড় করতে
গ) map মুছতে
ঘ) state লক করতে

১২৮. lookup(map, key, default) কী করে? [Intermediate]

ক) key মুছে দেয়
খ) map-এ key থাকলে তার মান, না থাকলে default দেয়
গ) map তৈরি করে
ঘ) key বদলায়

১২৯. try() function-এর কাজ কী? [Advanced]

ক) ত্রুটি হলে বিকল্প expression চেষ্টা করা
খ) লুপ চালানো
গ) state লক করা
ঘ) provider বদলানো

১৩০. coalesce() কী করে? [Intermediate]

ক) তালিকা সাজায়
খ) প্রথম non-empty/non-null মান ফেরত দেয়
গ) মান মুছে দেয়
ঘ) string জোড়া দেয়

১৩১. jsonencode() কী করে? [Intermediate]

ক) JSON পড়ে
খ) Terraform মানকে JSON string-এ রূপান্তর করে
গ) JSON মুছে দেয়
ঘ) state এনক্রিপ্ট করে

১৩২. templatefile() কী করে? [Intermediate]

ক) ফাইল মুছে দেয়
খ) টেমপ্লেট ফাইল পড়ে variable বসিয়ে string তৈরি করে

গ) provider ইনস্টল করে
ঘ) state লক করে

১৩৩. moved ব্লক কী করে? [Advanced]

ক) রিসোর্স মুছে দেয়
খ) ঠিকানা বদলালেও (যেমন module-এ সরালে) state ধরে রেখে recreation আটকায়
গ) state মুছে দেয়
ঘ) provider বদলায়

১৩৪. removed ব্লক কী করে? [Advanced]

ক) রিসোর্স তৈরি করে
খ) কনফিগারেশন থেকে সরানো রিসোর্স state থেকেও সরায়, বাস্তব রিসোর্স না মুছে
গ) state মুছে দেয়
ঘ) backend বদলায়

১৩৫. check ব্লক (Terraform 1.5+) কী করে? [Advanced]

ক) রিসোর্স তৈরি করে
খ) apply-এর পর assertion দিয়ে অবস্থা যাচাই করে
গ) state লক করে
ঘ) provider আপডেট করে

১৩৬. import ব্লক (Terraform 1.5+) কী সুবিধা দেয়? [Advanced]

ক) কোড ছাড়া import
খ) কনফিগারেশনে লিখে import করা যায়, ফলে পর্যালোচনাযোগ্য হয়
গ) state মুছে দেয়
ঘ) provider বদলায়

১৩৭. ephemeral value (Terraform 1.10+) কী? [Advanced]

ক) state-এ থাকে
খ) state-এ সংরক্ষণ না করা অস্থায়ী সংবেদনশীল মান
গ) output
ঘ) variable

১৩৮. write-only argument কী? [Advanced]

ক) state-এ থাকে
খ) শুধু লেখা যায়, state-এ সংরক্ষণ হয় না এমন argument
গ) output
ঘ) variable

সংক্ষিপ্ত প্রশ্ন • Short Answer Questions

সংক্ষিপ্ত প্রশ্ন

সংক্ষিপ্ত প্রশ্ন

মোট ৩০টি প্রশ্ন। প্রতিটি উত্তরে ৩–৬টি বাক্য লিখুন; ধারণা, উদাহরণ ও যুক্তি — তিনটিই থাকা চাই।

১. IaC বলতে কী বোঝায় এবং হাতে কনসোলে কাজ করার চেয়ে এর তিনটি সুবিধা লেখো। [Beginner]

২. Declarative ও imperative পদ্ধতির পার্থক্য একটি উদাহরণসহ ব্যাখ্যা করো। [Beginner]

৩. Terraform-এর state ফাইল কী এবং তা হারিয়ে গেলে কী সমস্যা হয়? [Beginner]

৪. terraform plan ও terraform apply-এর মধ্যে পার্থক্য লেখো। [Beginner]

৫. Remote backend কেন ব্যবহার করা হয়? S3 + DynamoDB উদাহরণ দাও। [Intermediate]

৬. State locking কী এবং force-unlock কখন নিরাপদ, কখন বিপজ্জনক? [Intermediate]

৭. count ও for_each-এর পার্থক্য লেখো; কখন কোনটি বেছে নেবে? [Intermediate]

৮. for_each-এর key বদলালে কেন resource recreate হয়? উদাহরণ দাও। [Intermediate]

৯. lifecycle ব্লকের তিনটি নিয়ম (prevent_destroy, create_before_destroy, ignore_changes) ব্যাখ্যা করো। [Intermediate]

১০. Module কী এবং একটি VPC module-এর input, output ও ব্যবহারের উদাহরণ দাও। [Intermediate]

১১. Module version pin না করলে কী ঝুঁকি হয়? [Intermediate]

১২. terraform import-এর ধাপগুলো লেখো; import-এর পর plan-এ পার্থক্য দেখালে কী করবে? [Intermediate]

১৩. Drift কীভাবে তৈরি হয় এবং তা শনাক্ত ও ঠিক করার উপায় লেখো। [Intermediate]

১৪. Workspace কী? একই কোডে dev ও prod আলাদা করার দুটি উপায় লেখো। [Intermediate]

১৫. Provider version constraint ~> 5.0 ও >= 5.0, < 6.0-এর পার্থক্য লেখো। [Intermediate]

১৬. .terraform.lock.hcl ফাইল কেন Git-এ রাখা উচিত? [Intermediate]

১৭. Sensitive variable ও output কীভাবে সামলাবে? State-এর ঝুঁকি কী? [Intermediate]

১৮. Dynamic block কখন ব্যবহার করা হয়? Security group ingress-এর উদাহরণ দাও। [Intermediate]

১৯. Terraform-এ secret রাখার নিরাপদ তিনটি উপায় লেখো; Git-এ secret কমিট করলে কী করবে? [Intermediate]

২০. Policy as Code কী? OPA বা Checkov দিয়ে একটি নিয়মের উদাহরণ দাও। [Advanced]

২১. CI/CD-তে Terraform workflow-এর ধাপগুলো লেখো (fmt, validate, plan, apply)। [Intermediate]

২২. terraform test ও Terratest-এর পার্থক্য লেখো। [Advanced]

২৩. Plan-এ অপ্রত্যাশিত destroy দেখলে তুমি কী কী যাচাই করবে? [Intermediate]

২৪. Apply মাঝপথে ব্যর্থ হলে কী করবে? State-এর অবস্থা কেমন থাকে? [Intermediate]

২৫. Module upgrade-এর পর resource address বদলে গেলে কীভাবে সামলাবে (moved/state mv)? [Advanced]

২৬. Backend বদলানোর নিরাপদ ধাপগুলো লেখো (migrate-state সহ)। [Advanced]

২৭. Cost explosion এড়াতে plan পর্যালোচনায় কী কী দেখবে? Infracost-এর ভূমিকা লেখো। [Intermediate]

২৮. Provisioner কেন শেষ উপায় হিসেবে ব্যবহার করা উচিত? বিকল্প কী? [Advanced]

২৯. terraform state mv ও state rm-এর পার্থক্য ও ব্যবহার লেখো। [Advanced]

৩০. Production-এ apply করার আগে তোমার pre-apply checklist-এ কী কী থাকবে? [Intermediate]

পরিস্থিতি • Troubleshooting Scenarios

সমস্যা সমাধানের পরিস্থিতি

সমস্যা সমাধানের পরিস্থিতি

মোট ২০টি পরিস্থিতি। প্রতিটিতে লক্ষণ, কী যাচাই করবেন ও সমাধানের পদ্ধতি সংক্ষেপে দেওয়া আছে; বিস্তারিত ব্যাখ্যা উত্তর-নির্দেশিকায় আছে।

১. State lock আটকে আছে

লক্ষণ: terraform apply চালালে "Error acquiring the state lock" দেখাচ্ছে; আগের apply অর্ধেক অবস্থায় বন্ধ হয়েছিল।

কী যাচাই করবেন: চলমান কোনো apply/process আছে কি না; lock-এর ID ও সময়; backend-এ lock রেকর্ড আছে কি না।

সমাধানের পদ্ধতি: কেউ apply করছে না নিশ্চিত হলে force-unlock; চলমান থাকলে অপেক্ষা করা।

২. Plan-এ অপ্রত্যাশিত destroy

লক্ষণ: কোনো পরিবর্তন না করেও plan-এ database destroy দেখাচ্ছে।

কী যাচাই করবেন: Git diff-এ নাম/argument বদলেছে কি না; module source/version বদলেছে কি না; state-এ resource আছে কি না।

সমাধানের পদ্ধতি: কারণ না খুঁজে apply নয়; নাম/ঠিকানা ঠিক করা বা moved ব্লক ব্যবহার করা।

৩. for_each key বদলে recreation

লক্ষণ: সার্ভারের নামের তালিকা বদলানোর পর সব instance recreate হতে চাইছে।

কী যাচাই করবেন: for_each-এ কী ব্যবহার হচ্ছে — নাম না স্থিতিশীল ID; key বদলেছে কি না।

সমাধানের পদ্ধতি: স্থিতিশীল key ব্যবহার করা; প্রয়োজনে state mv দিয়ে ঠিকানা মেলানো।

৪. Import mismatch

লক্ষণ: import করার পরও plan বড় পার্থক্য দেখাচ্ছে।

কী যাচাই করবেন: কনফিগারেশনের argument বাস্তব মানের সাথে মিলছে কি না; import করা ID ঠিক কি না।

সমাধানের পদ্ধতি: কনফিগারেশন বাস্তব অবস্থা অনুযায়ী মেলানো; তারপর plan শূন্য হওয়া যাচাই করা।

৫. Provider version conflict

লক্ষণ: init-এ "no available releases" বা version constraint ত্রুটি দেখাচ্ছে।

কী যাচাই করবেন: required_providers-এর constraint; lock ফাইলের সংস্করণ; registry-তে সংস্করণ আছে কি না।

সমাধানের পদ্ধতি: constraint শিথিল/আপডেট করা বা lock ফাইল পুনর্নির্মাণ করা।

৬. Backend auth ব্যর্থ

লক্ষণ: init-এ S3 backend-এ access denied দেখাচ্ছে।

কী যাচাই করবেন: credential/profile সঠিক কি না; bucket-এর IAM permission; region ঠিক কি না।

সমাধানের পদ্ধতি: সঠিক credential ও bucket policy ঠিক করে আবার init করা।

৭. State-এ sensitive মান ফাঁস

লক্ষণ: কেউ state ফাইল খুলে পাসওয়ার্ড দেখতে পেয়েছে।

কী যাচাই করবেন: state কোথায় সংরক্ষিত; কে access পায়; Git-এ state আছে কি না।

সমাধানের পদ্ধতি: secret ঘুরিয়ে (rotate) দেওয়া; backend encryption ও access কঠোর করা; Git থেকে state সরানো।

৮. Module upgrade ঠিকানা ভেঙেছে

লক্ষণ: Module version বাড়ানোর পর plan-এ সব resource destroy + create দেখাচ্ছে।

কী যাচাই করবেন: module-এর changelog; resource ঠিকানা/নাম বদলেছে কি না।

সমাধানের পদ্ধতি: moved ব্লক বা state mv দিয়ে ঠিকানা মেলানো; প্রয়োজনে পুরোনো সংস্করণে ফেরা।

৯. Console-এ হাতে বদলে drift

লক্ষণ: কনসোলে security group হাতে বদলানোর পর plan তা ফিরিয়ে নিতে চাইছে।

কী যাচাই করবেন: refresh-only plan-এ কী বদলেছে; কে, কখন বদলেছে।

সমাধানের পদ্ধতি: ইচ্ছাকৃত বদল হলে কনফিগারেশনে আনা; না হলে apply দিয়ে drift ঠিক করা।

১০. Count ভুলে খরচ বিস্ফোরণ

লক্ষণ: count-এ ভুল সংখ্যা দিয়ে শত শত instance তৈরি হয়ে গেছে।

কী যাচাই করবেন: plan-এ সংখ্যা যাচাই হয়েছিল কি না; count-এর মান কোথা থেকে এসেছে।

সমাধানের পদ্ধতি: তৎক্ষণাৎ apply থামানো; ভুল instance destroy করা; count-এ validation যোগ করা।

১১. Apply মাঝপথে ব্যর্থ

লক্ষণ: নেটওয়ার্ক ছিঁড়ে যাওয়ায় apply থেমেছে; কিছু resource তৈরি, কিছু হয়নি।

কী যাচাই করবেন: state-এ কোন resource তৈরি হয়েছে; ত্রুটির বার্তা।

সমাধানের পদ্ধতি: ত্রুটি ঠিক করে আবার plan/apply করা; state মুছে না দেওয়া।

১২. Lock timeout apply-এর সময়

লক্ষণ: বড় apply-এর সময় lock timeout হচ্ছে।

কী যাচাই করবেন: backend-এর lock সময়সীমা; নেটওয়ার্ক; দীর্ঘ apply।

সমাধানের পদ্ধতি: lock timeout বাড়ানো; apply ছোট ভাগে করা; backend স্বাস্থ্য যাচাই করা।

১৩. Workspace ভুলে prod-এ apply

লক্ষণ: dev ভেবে prod workspace-এ apply চালানো হয়েছে।

কী যাচাই করবেন: বর্তমান workspace (terraform workspace show); plan-এর resource নাম।

সমাধানের পদ্ধতি: তৎক্ষণাৎ পরিস্থিতি যাচাই; ভুল resource ফিরিয়ে আনা; workspace নাম prompt-এ দেখানো।

১৪. Secret Git-এ কমিট হয়েছে

লক্ষণ: AWS key .tf ফাইলে কমিট হয়ে গেছে।

কী যাচাই করবেন: কমিট history-তে key আছে কি না; key ব্যবহার হয়েছে কি না।

সমাধানের পদ্ধতি: key বাতিল (revoke) করা; history থেকে সরানো; secret manager-এ সরানো।

১৫. Destroy ভুলে production-এ

লক্ষণ: ভুল ডিরেক্টরি থেকে terraform destroy চালানো হয়েছে।

কী যাচাই করবেন: কোন workspace/backend-এ destroy চলেছে; state ব্যাকআপ আছে কি না।

সমাধানের পদ্ধতি: তৎক্ষণাৎ থামানো; ব্যাকআপ state থেকে পুনরুদ্ধারের চেষ্টা; prevent_destroy যোগ করা।

১৬. Provider upgrade breaking change

লক্ষণ: Provider আপডেটের পর argument আর চলে না।

কী যাচাই করবেন: changelog-এ removed/renamed argument; lock ফাইলের সংস্করণ।

সমাধানের পদ্ধতি: কনফিগারেশন নতুন argument অনুযায়ী ঠিক করা; test environment-এ যাচাই করা।

১৭. fmt/validate CI-তে ব্যর্থ

লক্ষণ: PR-এ fmt -check ও validate ব্যর্থ হচ্ছে।

কী যাচাই করবেন: কোন ফাইলে বিন্যাস ভুল; variable reference ভুল কি না।

সমাধানের পদ্ধতি: terraform fmt চালানো; ভুল reference ঠিক করা।

১৮. Data source পুরোনো মান দিচ্ছে

লক্ষণ: data source নতুন AMI না ধরে পুরোনোটি ধরছে।

কী যাচাই করবেন: data source-এর filter; dependency ঠিক আছে কি না।

সমাধানের পদ্ধতি: filter সুনির্দিষ্ট করা; depends_on দিয়ে ক্রম ঠিক করা।

১৯. Backend bucket মুছে গেছে

লক্ষণ: S3 backend bucket মুছে যাওয়ায় state পাওয়া যাচ্ছে না।

কী যাচাই করবেন: bucket-এর versioning/ব্যাকআপ ছিল কি না; local state ব্যাকআপ আছে কি না।

সমাধানের পদ্ধতি: versioning থেকে ফিরিয়ে আনা বা ব্যাকআপ state দিয়ে পুনর্নির্মাণ; ভবিষ্যতে versioning চালু করা।

২০. Output-এ sensitive মান দেখা যাচ্ছে

লক্ষণ: apply-এর পর পাসওয়ার্ড output-এ দেখা যাচ্ছে।

কী যাচাই করবেন: output-এ sensitive = true আছে কি না।

সমাধানের পদ্ধতি: sensitive = true যোগ করা; state access সীমিত করা; পাসওয়ার্ড rotate করা।

কেস স্টাডি • Production Incident Case Studies

উৎপাদন পরিবেশের ঘটনা

উৎপাদন পরিবেশের ঘটনা — কেস স্টাডি

মোট ১০টি কেস স্টাডি। প্রতিটি ঘটনা পড়ে নিজে কারণ বিশ্লেষণ করুন, তারপর উত্তর-নির্দেশিকার আলোচনার সাথে মিলিয়ে নিন।

১. State ফাইল মুছে যাওয়া

পটভূমি: একটি দলের local state ফাইল ভুলে মুছে গেছে; Terraform সব resource আবার তৈরি করতে চাইছে।

সময়রেখা: সোমবার সকাল: state মুছে যাওয়া ধরা পড়ে; plan-এ সব create দেখায়; দল apply থামায়।

প্রভাব: Production resource ডুপ্লিকেট হওয়ার ঝুঁকি; downtime-এর আশঙ্কা।

মূল কারণ বিশ্লেষণের প্রশ্ন: কেন local state ব্যবহার করা হয়েছিল? ব্যাকআপ ছিল কি না? Remote backend থাকলে কী বদলাত?

শিক্ষা: Remote backend, versioning ও ব্যাকআপের গুরুত্ব।

২. Concurrent apply-তে state নষ্ট

পটভূমি: দুজন একই সময়ে apply চালিয়ে state corrupt করেছে।

সময়রেখা: দুটি apply একই সময়ে শুরু; একটির লেখা অন্যটি মুছে দেয়; state পড়া যাচ্ছে না।

প্রভাব: State পড়া যাচ্ছে না; পরের apply আটকে গেছে।

মূল কারণ বিশ্লেষণের প্রশ্ন: Locking কেন কাজ করেনি? Local backend-এ lock ছিল কি না?

শিক্ষা: Locking ছাড়া দলগত কাজ বিপজ্জনক; remote backend বাধ্যতামূলক।

৩. Secret Git-এ কমিট

পটভূমি: AWS access key Terraform ফাইলে কমিট হয়ে public repo-তে উঠেছে।

সময়রেখা: কমিটের কয়েক ঘণ্টা পর bot key ব্যবহার করে resource তৈরি করে।

প্রভাব: অননুমোদিত বিল ও resource তৈরি।

মূল কারণ বিশ্লেষণের প্রশ্ন: কেন key কোডে ছিল? কীভাবে ধরা পড়ল? কীভাবে key বাতিল করা হলো?

শিক্ষা: Secret manager, .gitignore, pre-commit secret scan।

৪. Production-এ ভুল destroy

পটভূমি: ভুল workspace থেকে destroy চালিয়ে production database মুছে গেছে।

সময়রেখা: Destroy চালানোর ৫ মিনিটে ডাটাবেস মুছে যায়; অ্যাপ বন্ধ হয়ে যায়।

প্রভাব: সেবা বন্ধ; ডেটা হারানোর ঝুঁকি।

মূল কারণ বিশ্লেষণের প্রশ্ন: prevent_destroy ছিল কি না? কেন workspace যাচাই হয়নি? ব্যাকআপ ছিল কি না?

শিক্ষা: prevent_destroy, ব্যাকআপ, workspace যাচাই ও অনুমোদন প্রক্রিয়া।

৫. Module version bump-এ database ধ্বংস

পটভূমি: Module আপডেটের পর database resource-এর নাম বদলে destroy + create দেখায়; কেউ apply করে দেয়।

সময়রেখা: Apply-এর পর পুরোনো database মুছে নতুন খালি database তৈরি হয়।

প্রভাব: ডেটা হারানো; সেবা বন্ধ।

মূল কারণ বিশ্লেষণের প্রশ্ন: Changelog পড়া হয়েছিল কি না? Plan-এ destroy কেন উপেক্ষা করা হলো?

শিক্ষা: Plan পর্যালোচনা, module pin, moved ব্লক।

৬. Drift-এ outage

পটভূমি: কনসোলে হাতে load balancer বদলানো হয়েছিল; পরের apply তা ফিরিয়ে নিয়ে outage ঘটায়।

সময়রেখা: Apply-এর সাথে সাথে load balancer পুরোনো কনফিগারেশনে ফিরে যায়।

প্রভাব: সেবা কয়েক মিনিট বন্ধ।

মূল কারণ বিশ্লেষণের প্রশ্ন: Drift কেন শনাক্ত হয়নি? হাতে বদল কেন হয়েছিল?

শিক্ষা: Drift detection, refresh-only plan, কনসোলে হাতে বদল নিষিদ্ধ করা।

৭. Backend bucket মুছে যাওয়া

পটভূমি: পরিষ্কার করতে গিয়ে S3 backend bucket মুছে ফেলা হয়।

সময়রেখা: পরের plan-এ state পাওয়া যাচ্ছে না; দল আটকে যায়।

প্রভাব: সব environment-এর কাজ থেমে যায়।

মূল কারণ বিশ্লেষণের প্রশ্ন: Bucket-এ versioning ও deletion protection ছিল কি না? ব্যাকআপ কোথায়?

শিক্ষা: Backend bucket-এ versioning, MFA delete, ব্যাকআপ।

৮. Lock timeout-এ apply ব্যর্থ

পটভূমি: বড় apply-এর সময় lock timeout হয়ে apply মাঝপথে থামে।

সময়রেখা: Lock timeout; আংশিক resource তৈরি; দল আবার apply করে।

প্রভাব: অসম্পূর্ণ অবকাঠামো; পুনরায় apply-এ সময় নষ্ট।

মূল কারণ বিশ্লেষণের প্রশ্ন: কেন lock timeout হলো? Apply কি ভাগ করা যেত?

শিক্ষা: ছোট apply, lock timeout কনফিগারেশন, backend স্বাস্থ্য।

৯. Provider upgrade-এ breaking change

পটভূমি: Provider major version আপডেটের পর কনফিগারেশন আর চলে না; production apply আটকে যায়।

সময়রেখা: CI-তে validate ব্যর্থ; দল rollback করে।

প্রভাব: Release বিলম্বিত।

মূল কারণ বিশ্লেষণের প্রশ্ন: Changelog পড়া হয়েছিল কি না? Test environment-এ যাচাই হয়েছিল কি না?

শিক্ষা: Version pin, changelog, test environment।

১০. Count ভুলে খরচ বিস্ফোরণ

পটভূমি: Variable-এ ভুল মান দিয়ে ২০০টি VM তৈরি হয়ে যায়; বিল হাজার ডলার ছাড়ায়।

সময়রেখা: Apply-এর পর বিল অ্যালার্ট আসে; দল instance মুছে দেয়।

প্রভাব: অপ্রত্যাশিত বড় বিল।

মূল কারণ বিশ্লেষণের প্রশ্ন: Plan-এ সংখ্যা যাচাই হয়েছিল কি না? Cost check ছিল কি না?

শিক্ষা: Plan পর্যালোচনা, Infracost, budget alert, validation।

চূড়ান্ত পরীক্ষা • Final Practical Examination

চূড়ান্ত ব্যবহারিক পরীক্ষা

চূড়ান্ত ব্যবহারিক পরীক্ষা

সময়: ৩ ঘণ্টা (১৮০ মিনিট) • **মোট নম্বর:** ১০০ • **পাস নম্বর:** ৬০ • **পরিবেশ:** নিজের ল্যাপটপে test ক্লাউড অ্যাকাউন্ট বা localstack-সদৃশ test পরিবেশ; production অ্যাকাউন্ট ব্যবহার করা যাবে না।

পরীক্ষা চলাকালীন বই ও নিজের নোট দেখা যাবে, কিন্তু অন্যের সমাধান কপি করা যাবে না। প্রতিটি কাজের পর `terraform plan`-এর আউটপুট সংরক্ষণ করুন — মূল্যায়নে plan পড়ার দক্ষতাও দেখা হবে।

কাজ	বিবরণ	সময়	নম্বর
১. মডুলার অবকাঠামো	network, compute ও storage — তিনটি module দিয়ে একটি মডুলার অবকাঠামো তৈরি করুন; VPC, subnet, security group, একটি VM/instance ও একটি storage bucket থাকবে। প্রতিটি module-এ variable, output ও README থাকবে।	৬০ মিনিট	৩০
২. বহু পরিবেশ	একই কোডে dev ও prod পরিবেশ আলাদা করুন — আলাদা tfvars ফাইল ও আলাদা backend key/state ব্যবহার করে। দুই পরিবেশের পার্থক্য (যেমন instance আকার, tag) দেখান।	৩০ মিনিট	২০
৩. Remote backend	Remote backend কনফিগার করুন (S3 + DynamoDB, azurerm বা gcs — যেকোনো একটি); versioning ও encryption চালু আছে তা দেখান এবং locking কাজ করছে তা প্রমাণ করুন।	৩০ মিনিট	১৫
৪. Import	হাতে তৈরি একটি বিদ্যমান resource-কে import করে Terraform-এর ব্যবস্থাপনায় আনুন; import-এর পর plan-এ “No changes” দেখান।	৩০ মিনিট	১৫
৫. CI plan পর্যালোচনা	একটি পরিবর্তনের PR-এর জন্য fmt, validate ও plan চালিয়ে পর্যালোচনা-মন্তব্য লিখুন — কী বদলাবে, কোনো destroy/replace আছে কি না, খরচের প্রভাব কী।	৩০ মিনিট	২০

পাসের মানদণ্ড

পাস করতে হলে মোট ৬০ নম্বরের পাশাপাশি কাজ ১ ও কাজ ৪-এ আলাদাভাবে কমপক্ষে অর্ধেক নম্বর পেতে হবে। কোনো কাজে production-সমতুল্য পরিবেশে অপ্রত্যাশিত destroy করলে সেই কাজে শূন্য ধরা হবে। পরীক্ষার শেষে সব resource `terraform destroy` দিয়ে মুছে ফেলুন এবং destroy-এর plan সংরক্ষণ করুন।

> **সতর্কতা:** পরীক্ষার কাজগুলো শুধু test পরিবেশে করুন। ভুল workspace বা ভুল backend-এ apply করলে আসল অবকাঠামো নষ্ট হতে পারে — প্রতিটি apply-এর আগে workspace ও backend যাচাই করুন।

উত্তর-নির্দেশিকা (প্রথম অংশ) • Answer Guide Part 1

উত্তর-নির্দেশিকা — প্রথম অংশ

উত্তর-নির্দেশিকা — প্রথম অংশ

এই অংশে বহুনির্বাচনি প্রশ্ন ১–৬৯, সংক্ষিপ্ত প্রশ্ন ১–১৫, পরিস্থিতি ১–১০ ও কেস স্টাডি ১–৫-এর উত্তর আছে।

বহুনির্বাচনি প্রশ্নের উত্তর (১–৬৯)

১. সঠিক উত্তর: খ) অবকাঠামোকে সংস্করণ-নিয়ন্ত্রিত কোড হিসেবে সংজ্ঞায়িত ও পরিচালনা করা [Beginner]
IaC-তে অবকাঠামো কোডে লেখা হয়, Git-এ সংস্করণ নিয়ন্ত্রণ করা যায় এবং বারবার একইভাবে তৈরি করা যায়।

২. সঠিক উত্তর: খ) শেষ অবস্থা (desired state) ঘোষণা করা; টুল কীভাবে পৌঁছাবে তা ঠিক করে [Beginner]
Declarative-তে আপনি চূড়ান্ত অবস্থা বলেন; Terraform সেই অবস্থায় পৌঁছানোর পরিকল্পনা নিজে তৈরি করে।

৩. সঠিক উত্তর: খ) Ansible playbook-এ ধাপে ধাপে কমান্ড দেওয়া নয় — আসলে Bash স্ক্রিপ্টে create → configure ধাপ লেখা [Beginner]
Imperative-তে কীভাবে করতে হবে তার ধাপ লেখা হয়, যেমন Bash স্ক্রিপ্টে সার্ভার তৈরি করে তারপর কনফিগার করা।

৪. সঠিক উত্তর: খ) বিভিন্ন ক্লাউডে অবকাঠামো provision ও পরিচালনা করা [Beginner]
Terraform provider-এর মাধ্যমে ক্লাউড রিসোর্স তৈরি, পরিবর্তন ও ধ্বংস করে।

৫. সঠিক উত্তর: ক) HashiCorp Configuration Language [Beginner]
HCL হলো HashiCorp Configuration Language — Terraform-এর কনফিগারেশন ভাষা।

৬. সঠিক উত্তর: খ) পরিচালনা করতে হবে এমন একটি অবকাঠামো উপাদান [Beginner]
resource ব্লক Terraform-কে বলে কোন অবকাঠামো তৈরি ও পরিচালনা করতে হবে।

৭. সঠিক উত্তর: খ) বিদ্যমান অবকাঠামো বা বাইরের তথ্য পড়ে ব্যবহার করা [Beginner]
data source Terraform পরিচালনা করে না; শুধু বিদ্যমান তথ্য পড়ে কনফিগারেশনে ব্যবহার করে।

৮. সঠিক উত্তর: খ) provider plugin ডাউনলোড করে ও backend কনফিগার করে [Beginner]
init backend ও provider plugin প্রস্তুত করে; না চালালে plan/apply চলবে না।

৯. সঠিক উত্তর: খ) পরিবর্তন প্রয়োগ না করে কী বদলাবে তা দেখানো [Beginner]
plan একটি execution plan দেখায় — কী তৈরি, বদল বা ধ্বংস হবে — প্রয়োগ না করেই।

১০. সঠিক উত্তর: খ) পরিকল্পিত পরিবর্তন বাস্তবে প্রয়োগ করে [Beginner]
apply অনুমোদনের পর plan অনুযায়ী রিসোর্স তৈরি/পরিবর্তন/ধ্বংস করে ও state আপডেট করে।

১১. সঠিক উত্তর: খ) বর্তমান অবকাঠামোর অবস্থা ও রিসোর্স-কনফিগারেশন mapping [Beginner]
state-এ রিসোর্সের বর্তমান অবস্থা, ID, attribute ও dependency থাকে।

১২. সঠিক উত্তর: খ) Terraform বিদ্যমান রিসোর্স চিনতে পারে না; পুনরায় তৈরি করতে চাইবে [Intermediate]
state ছাড়া Terraform জানে না রিসোর্স আছে; তাই আবার তৈরি করার plan দেখায় বা import করতে হয়।

১৩. সঠিক উত্তর: খ) দলগতভাবে একই state শেয়ার ও লক করা যায় [Beginner]
Remote backend (যেমন S3) দলকে একই state দেয় ও concurrent apply থেকে রক্ষা করে।

১৪. সঠিক উত্তর: খ) একই সময়ে দুটি apply যাতে state নষ্ট না করে [Intermediate]
Locking একসাথে একাধিক লেখা আটকায়; না থাকলে state corrupt হতে পারে।

১৫. সঠিক উত্তর: খ) কনফিগারেশনের সব পরিচালিত রিসোর্স ধ্বংস করে [Beginner]
destroy কনফিগারেশন অনুযায়ী সব managed resource মুছে দেয় — production-এ অত্যন্ত ঝুঁকিপূর্ণ।

১৬. সঠিক উত্তর: খ) কনফিগারেশনকে পুনর্ব্যবহারযোগ্য করতে ইনপুট মান নেওয়া [Beginner]
variable ইনপুট প্যারামিটার দেয়; environment ভেদে মান বদলানো যায়।

১৭. সঠিক উত্তর: খ) তৈরি রিসোর্সের গুরুত্বপূর্ণ মান (যেমন IP, ARN) প্রকাশ করতে [Beginner]
output apply-এর পর মান দেখায় ও অন্য মডিউল/সিস্টেম ব্যবহার করতে পারে।

১৮. সঠিক উত্তর: খ) একই expression বারবার না লিখে স্থানীয়ভাবে নাম দিয়ে ব্যবহার করা [Beginner]
locals জটিল expression-কে একবার গণনা করে নাম দিয়ে পুনর্ব্যবহার করে।

১৯. সঠিক উত্তর: খ) primitive type [Beginner]
এগুলো primitive type; list/map/object হলো collection/structural type।

২০. সঠিক উত্তর: খ) list-এ ক্রম ও ডুপ্লিকেট থাকে, set-এ ক্রম নেই ও ডুপ্লিকেট নেই [Intermediate]
list ordered ও ডুপ্লিকেট রাখে; set unordered ও unique মান রাখে।

২১. সঠিক উত্তর: খ) key-value জোড়ার সংগ্রহ [Beginner]
map-এ প্রতিটি key-এর সাথে একটি মান থাকে, যেমন tags।

২২. সঠিক উত্তর: ক) যখন বিভিন্ন attribute-এর নির্দিষ্ট কাঠামো দরকার [Intermediate]
object নির্দিষ্ট attribute ও টাইপের কাঠামো নির্ধারণ করে, যেমন একটি সার্ভারের spec।

২৩. সঠিক উত্তর: খ) apply-এর সময় মান দিতে হয়, না দিলে ত্রুটি হয় [Beginner]
required variable-এর মান না দিলে Terraform মান চায় বা ত্রুটি দেয়।

২৪. সঠিক উত্তর: খ) CLI আউটপুটে মান দেখানো বন্ধ করে (তবে state-এ থাকে) [Intermediate]
sensitive আউটপুটে মান গোপন করে, কিন্তু state ফাইলে মান থাকে — তাই state সুরক্ষিত রাখতে হয়।

২৫. সঠিক উত্তর: খ) যখন implicit dependency যথেষ্ট নয়, তখন স্পষ্ট নির্ভরতা বোঝাতে [Intermediate]
Terraform সাধারণত reference থেকে dependency বোঝে; hidden dependency থাকলে depends_on দিতে হয়।

২৬. সঠিক উত্তর: খ) একটি রিসোর্সের attribute অন্য রিসোর্সে ব্যবহার করলে [Beginner]
একটি রিসোর্সের মান অন্য রিসোর্সে উল্লেখ করলে Terraform নিজে ক্রম ঠিক করে।

২৭. সঠিক উত্তর: খ) একই রিসোর্সের তিনটি instance তৈরি হয় [Beginner]
count একই রিসোর্সের একাধিক instance তৈরি করে, index দিয়ে চেনা যায়।

২৮. সঠিক উত্তর: খ) ০ থেকে শুরু হওয়া ক্রমিক সংখ্যা [Beginner]
count.index ০, ১, ২... হয়; নামকরণে ব্যবহার করা যায়।

২৯. সঠিক উত্তর: খ) যখন প্রতিটি instance-কে স্থিতিশীল key (যেমন নাম) দিয়ে চেনা দরকার [Intermediate]
for_each map/set-এর key দিয়ে instance চেনে; মাঝের একটি মুছলে অন্যগুলো পুনরায় তৈরি হয় না।

৩০. সঠিক উত্তর: খ) পুরোনো key-এর instance ধ্বংস হয়ে নতুন key-এর instance তৈরি হয় [Intermediate]
for_each instance key দিয়েই চেনা হয়; key বদল মানে পুরোনোটি ধ্বংস ও নতুনটি তৈরি।

৩১. সঠিক উত্তর: খ) for_each-এর বর্তমান instance-এর key ও মান [Beginner]
for_each ব্লকে each.key বর্তমান key ও each.value বর্তমান মান দেয়।

৩২. সঠিক উত্তর: খ) ধ্বংস করার plan-কে ত্রুটি দিয়ে আটকায় [Intermediate]
prevent_destroy ধ্বংস-পরিকল্পনা আটকায়; database-এর মতো গুরুত্বপূর্ণ রিসোর্সে ব্যবহার করা হয়।

৩৩. সঠিক উত্তর: ক) যখন downtime এড়িয়ে আগে নতুনটি তৈরি করে পরে পুরোনোটি মুছতে হয় [Intermediate]
এটি replacement-এর সময় প্রথমে নতুন রিসোর্স তৈরি করে, তারপর পুরোনোটি মুছে downtime কমায়।

৩৪. সঠিক উত্তর: খ) নির্দিষ্ট attribute-এর বাইরের পরিবর্তন plan-এ দেখায় না [Intermediate]
ignore_changes নির্দিষ্ট attribute-এ manual/বাইরের পরিবর্তন Terraform-কে উপেক্ষা করতে বলে।

৩৫. সঠিক উত্তর: খ) পরের apply-তে রিসোর্সটি ধ্বংস করে পুনরায় তৈরি করতে চিহ্নিত করে [Intermediate]
taint/replace চিহ্নিত রিসোর্স পরের apply-তে recreate হয়; দূষিত রিসোর্স ঠিক করতে ব্যবহৃত হয়।

৩৬. সঠিক উত্তর: ক) যখন nested block-এর সংখ্যা variable-এর ওপর নির্ভর করে [Intermediate]
dynamic block for_each দিয়ে পুনরাবৃত্ত nested block তৈরি করে, যেমন একাধিক ingress rule।

৩৭. সঠিক উত্তর: খ) merge(), length(), format() [Beginner]
merge, length, format প্রভৃতি built-in function মান গণনা ও রূপান্তরে ব্যবহার হয়।

৩৮. সঠিক উত্তর: খ) web-01 [Beginner]
format %s স্থানগুলোতে মান বসিয়ে web-01 তৈরি করে।

৩৯. সঠিক উত্তর: খ) ২ [Beginner]
তালিকায় দুটি উপাদান আছে, তাই length ২।

৪০. সঠিক উত্তর: খ) পুনর্ব্যবহারযোগ্য Terraform কনফিগারেশনের সংগ্রহ [Beginner]
Module একসাথে ব্যবহৃত resource-এর প্যাকেজ; বারবার ব্যবহার করা যায়।

৪১. সঠিক উত্তর: খ) মডিউলের অবস্থান — local path, Git বা Registry [Beginner]
source বলে মডিউল কোথা থেকে আসবে — ./modules/vpc, Git URL বা registry ঠিকানা।

৪২. সঠিক উত্তর: খ) অপ্রত্যাশিত breaking change থেকে রক্ষা পেতে নির্দিষ্ট সংস্করণ ব্যবহার করা যায় [Intermediate]
version pin না করলে নতুন মডিউল সংস্করণ হঠাৎ breaking change আনতে পারে।

৪৩. সঠিক উত্তর: খ) provider ও module খোঁজা ও ব্যবহারের সরকারি ভান্ডার [Beginner]
Registry-তে যাচাই করা provider ও community module পাওয়া যায়।

৪৪. সঠিক উত্তর: খ) module.<নাম>.<output> দিয়ে [Intermediate]
Child module output ঘোষণা করলে parent module.<নাম>.<output> দিয়ে তা ব্যবহার করে।

৪৫. সঠিক উত্তর: খ) বিদ্যমান বাস্তব রিসোর্সকে state-এ নিয়ে আসে [Intermediate]
import হাতে তৈরি রিসোর্সকে Terraform-এর ব্যবস্থাপনায় আনে; কনফিগারেশন নিজে লিখতে হয়।

৪৬. সঠিক উত্তর: খ) কনফিগারেশন বাস্তব অবস্থার সাথে মেলেনি; কনফিগারেশন মেলাতে হবে [Intermediate]
import শুধু state-এ আনে; কনফিগারেশন বাস্তব অবস্থা না মেলালে plan পরিবর্তন দেখাবে।

৪৭. সঠিক উত্তর: খ) কনফিগারেশনের বাইরে বাস্তব অবকাঠামোতে পরিবর্তন [Intermediate]
কনসোলে হাতে বদলালে বাস্তব অবস্থা কনফিগারেশন থেকে সরে যায় — সেটাই drift।

৪৮. সঠিক উত্তর: খ) বাস্তব অবস্থা পড়ে state আপডেট করে [Intermediate]
refresh-only বাস্তব অবস্থা পড়ে state মেলায়, drift দেখতে সাহায্য করে।

৪৯. সঠিক উত্তর: খ) একই কনফিগারেশন দিয়ে একাধিক environment-এর আলাদা state [Intermediate]
workspace একই কোডে dev/staging/prod-এর আলাদা state রাখে।

৫০. সঠিক উত্তর: খ) prod নামে নতুন workspace ও খালি state তৈরি করে [Beginner]
এটি prod workspace তৈরি করে; এর state অন্যগুলো থেকে আলাদা থাকে।

৫১. সঠিক উত্তর: খ) 5.x সংস্করণ (5.0 থেকে 6.0-এর আগে) [Intermediate]
~>-এর ডানদিকের শেষ অংশ বদলাতে দেয়; ~> 5.0 মানে 5.x অনুমোদিত, 6.0 নয়।

৫২. সঠিক উত্তর: খ) provider-এর নির্দিষ্ট সংস্করণ ও hash লক করা [Intermediate]
lock ফাইল নিশ্চিত করে দলের সবাই একই provider সংস্করণ ব্যবহার করবে।

৫৩. সঠিক উত্তর: খ) কোডের বিন্যাস একরকম করে [Beginner]
fmt HCL-এর indentation ও বিন্যাস ঠিক করে; CI-তে fmt -check দিয়ে যাচাই করা যায়।

৫৪. সঠিক উত্তর: খ) কনফিগারেশনের syntax ও reference ঠিক আছে কি না [Beginner]
validate syntax ও internal reference যাচাই করে; ক্লাউডে কিছু তৈরি করে না।

৫৫. সঠিক উত্তর: খ) DynamoDB টেবিল [Intermediate]
S3 backend-এ DynamoDB টেবিল lock ধারণ করে; নতুন সংস্করণে S3 native locking-ও আছে।

৫৬. সঠিক উত্তর: খ) versioning ও encryption চালু করা [Intermediate]
Versioning ভুল state ফিরিয়ে আনতে দেয়, encryption সংবেদনশীল মান রক্ষা করে।

৫৭. সঠিক উত্তর: খ) Azure Storage Account-এর Blob-এ [Intermediate]
azurerm backend Azure Blob Storage-এ state রাখে ও lease দিয়ে lock করে।

৫৮. সঠিক উত্তর: খ) Google Cloud Storage bucket-এ [Beginner]
gcs backend Google Cloud Storage bucket-এ state রাখে।

৫৯. সঠিক উত্তর: ক) Git-এ plain text-এ [Beginner]
Git-এ plain text secret কমিট করা মারাত্মক ভুল; secret manager থেকে পড়া উচিত।

৬০. সঠিক উত্তর: খ) state পড়তে পারলে secret ফাঁস হয় [Intermediate]
state-এ প্রায়ই secret-এর plain মান থাকে; তাই backend encryption ও কঠোর access দরকার।

৬১. সঠিক উত্তর: খ) নিয়ম (যেমন encryption বাধ্যতামূলক) কোডে লিখে স্বয়ংক্রিয়ভাবে যাচাই করা [Intermediate]
OPA/Sentinel দিয়ে নিয়ম কোডে লেখা হয় ও plan-এর সময় যাচাই করা হয়।

৬২. সঠিক উত্তর: খ) Rego নীতিমালা দিয়ে plan যাচাই করা [Advanced]
OPA Rego ভাষায় লেখা নীতি দিয়ে Terraform plan মূল্যায়ন করে।

৬৩. সঠিক উত্তর: খ) Terraform কনফিগারেশনে নিরাপত্তা ও compliance ত্রুটি [Intermediate]
Checkov static analysis করে — যেমন খোলা security group বা encryption ছাড়া storage।

৬৪. সঠিক উত্তর: খ) Terraform কোডে ভুল, deprecated syntax ও provider নিয়ম যাচাই করে [Intermediate]
TFLint linting করে; provider plugin দিয়ে গভীর নিয়মও যাচাই করা যায়।

৬৫. সঠিক উত্তর: খ) pull request-এ পরিবর্তন পর্যালোচনা করতে [Beginner]
CI-তে plan-এর আউটপুট PR-এ দেখিয়ে পর্যালোচনা করা হয়, apply শুধু merge-এর পর হয়।

৬৬. সঠিক উত্তর: খ) plan ফাইল সংরক্ষণ করে পরে একই plan apply করা যায় [Intermediate]
সংরক্ষিত plan দিয়ে নিশ্চিত করা যায় পর্যালোচিত plan-ই apply হবে।

৬৭. সঠিক উত্তর: খ) অপ্রত্যাশিত destroy বা replacement ধরা পড়ে [Beginner]
plan-এ destroy/replace চোখে পড়লে ভুল apply আটকানো যায়।

৬৮. সঠিক উত্তর: খ) hashicorp/aws [Beginner]
AWS রিসোর্সের জন্য hashicorp/aws provider ব্যবহৃত হয়।

৬৯. সঠিক উত্তর: খ) প্রয়োজনীয় argument — কোন image ও কোন আকারের সার্ভার [Beginner]
ami কোন image থেকে সার্ভার তৈরি হবে ও instance_type তার আকার নির্ধারণ করে।

সংক্ষিপ্ত প্রশ্নের উত্তর (১–১৫)

১. IaC মানে অবকাঠামোকে সংস্করণ-নিয়ন্ত্রিত কোড হিসেবে লেখা ও পরিচালনা করা। সুবিধা: (১) একই কোড বারবার চালিয়ে একই অবকাঠামো তৈরি করা যায়; (২) Git-এ পরিবর্তনের ইতিহাস থাকে ও পর্যালোচনা করা যায়; (৩) ভুল হলে আগের সংস্করণে ফেরা যায়।

২. Declarative-তে শেষ অবস্থা ঘোষণা করা হয় (যেমন Terraform-এ ‘একটি t3.micro instance চাই’), আর imperative-তে ধাপ লেখা হয় (যেমন Bash স্ক্রিপ্টে instance তৈরি করে তারপর কনফিগার করা)। Declarative-তে টুল নিজে পার্থক্য বের করে; imperative-তে ক্রমের দায়িত্ব লেখকের।

৩. State ফাইল Terraform-এর রেকর্ড — কোন resource বাস্তবে আছে, তার ID ও attribute কী, তা এতে থাকে। হারিয়ে গেলে Terraform বিদ্যমান resource চিনতে পারে না, আবার তৈরি করার plan দেখায়; তাই remote backend, versioning ও ব্যাকআপ জরুরি।

৪. plan শুধু কী বদলাবে তা দেখায়, কিছু প্রয়োগ করে না; apply অনুমোদনের পর সেই পরিবর্তন বাস্তবে প্রয়োগ করে ও state আপডেট করে। ভালো অভ্যাস হলো plan পর্যালোচনা করে তারপর apply করা।

৫. Remote backend দলের সবাইকে একই state দেয় ও locking করে। S3-তে state রাখা হয়, DynamoDB টেবিল lock ধারণ করে; S3-তে versioning ও encryption চালু রাখলে ভুল state ফিরিয়ে আনা যায় ও secret সুরক্ষিত থাকে।

৬. State locking একসাথে দুটি apply-কে state লেখা থেকে আটকায়। কোনো apply চলছে না নিশ্চিত হলে আটকে থাকা lock force-unlock করা নিরাপদ; চলমান apply-এর lock খুললে state corrupt হতে পারে, তাই আগে প্রক্রিয়া যাচাই করতে হয়।

৭. count সংখ্যা দিয়ে instance তৈরি করে (index ০, ১, ২), for_each map/set-এর key দিয়ে তৈরি করে। মাঝের একটি মুছলে count-এ index সরে replace হতে পারে; for_each-এ key স্থিতিশীল থাকলে অন্য instance অক্ষত থাকে। নাম দিয়ে চেনা দরকার হলে for_each ভালো।

৮. for_each instance-কে key দিয়ে চেনে। Key বদলালে Terraform পুরোনো key-এর instance-কে নতুন বলে চেনে না — পুরোনোটি destroy ও নতুন key-এরটি create হয়। তাই key হিসেবে স্থিতিশীল পরিচয় (যেমন সার্ভারের নাম যা বদলাবে না) ব্যবহার করা উচিত।

৯. prevent_destroy ধ্বংসের plan আটকায়; create_before_destroy replacement-এর সময় আগে নতুনটি তৈরি করে downtime কমায়; ignore_changes নির্দিষ্ট attribute-এর বাইরের পরিবর্তন উপেক্ষা করে। তিনটিই ঝুঁকি কমাতে ব্যবহৃত হয়, তবে ignore_changes-এ drift লুকাতে পারে বলে সাবধানে ব্যবহার করতে হয়।

১০. Module হলো পুনর্ব্যবহারযোগ্য কনফিগারেশনের প্যাকেজ। VPC module-এ input হিসেবে CIDR ও পরিবেশের নাম, ভেতরে VPC/subnet/route table resource, আর output হিসেবে VPC ID ও subnet ID থাকতে পারে; root module source দিয়ে ডেকে output ব্যবহার করে।

১১. Version pin না করলে module-এর নতুন সংস্করণ হঠাৎ breaking change আনতে পারে — argument বদলে যাওয়া বা resource-এর নাম বদলে destroy হতে পারে। নির্দিষ্ট সংস্করণ pin করলে কখন আপডেট হবে তা দলের নিয়ন্ত্রণে থাকে।

১২. ধাপ: (১) বাস্তব resource-এর ID দিয়ে terraform import করা; (২) সেই resource-এর কনফিগারেশন লেখা; (৩) plan চালিয়ে ‘No changes’ যাচাই করা। পার্থক্য দেখালে কনফিগারেশন বাস্তব মানের সাথে মেলাতে হয় — import শুধু state-এ আনে, কনফিগারেশন নিজে লিখতে হয়।

১৩. Drift তৈরি হয় কনসোলে হাতে বদলালে বা বাইরের সিস্টেম বদলালে। refresh-only plan দিয়ে শনাক্ত করা যায়; ইচ্ছাকৃত বদল হলে কনফিগারেশনে আনা উচিত, না হলে apply দিয়ে বাস্তব অবস্থা কনফিগারেশনের সাথে মেলানো হয়।

১৪. Workspace একই কনফিগারেশনে আলাদা state রাখে — যেমন dev ও prod। বিকল্প: আলাদা ডিরেক্টরি বা আলাদা tfvars ফাইল ও backend key ব্যবহার করা। বড় দলে আলাদা ডিরেক্টরি/tfvars পদ্ধতিই বেশি ব্যবহৃত হয়।

১৫. ~> 5.0 মানে 5.x অনুমোদিত, 6.0 নয়; >= 5.0, < 6.0 একই সীমা স্পষ্টভাবে লেখে। ~> শুধু ডানদিকের শেষ অংশ বদলাতে দেয় — ~> 5.2 মানে 5.2 থেকে 6.0-এর আগে পর্যন্ত।

পরিস্থিতির সমাধান (১–১০)

১. প্রথমে চলমান apply/process আছে কি না যাচাই করুন। না থাকলে lock-এর ID ও সময় দেখে terraform force-unlock করুন; চলমান থাকলে অপেক্ষা করুন। বারবার আটকালে backend-এর স্বাস্থ্য ও নেটওয়ার্ক যাচাই করুন।

২. apply থামান। Git diff, module সংস্করণ ও resource-এর নাম/argument বদলেছে কি না দেখুন। কারণ ঠিক না করে apply করবেন না; ঠিকানা বদলে থাকলে moved ব্লক বা state mv ব্যবহার করুন।

৩. for_each-এ অস্থিতিশীল key (যেমন পরিবর্তনশীল নাম) ব্যবহার হয়েছে কি না দেখুন। স্থিতিশীল key ব্যবহার করুন; ইতিমধ্যে বদলে গেলে state mv দিয়ে পুরোনো ঠিকানা নতুন key-এর সাথে মেলান।

৪. import শুধু state-এ আনে; কনফিগারেশন বাস্তব মানের সাথে মেলাতে হয়। terraform show দিয়ে বাস্তব attribute দেখে কনফিগারেশন ঠিক করুন, তারপর plan-এ ‘No changes’ যাচাই করুন।

৫. required_providers-এর constraint ও lock ফাইল যাচাই করুন। constraint খুব কঠোর হলে শিথিল করুন, অথবা terraform init -upgrade দিয়ে অনুমোদিত নতুন সংস্করণ নিন।

৬. credential, profile, region ও bucket-এর IAM permission যাচাই করুন। সঠিক credential দিয়ে আবার init করুন; প্রয়োজনে backend-config দিয়ে মান দিন।

৭. তৎক্ষণাৎ সংশ্লিষ্ট secret rotate (বদলে) দিন। state কোথায় আছে দেখুন, Git থেকে সরান, backend-এ encryption চালু করুন ও access সীমিত করুন।

৮. module-এর changelog পড়ুন — resource-এর নাম বা ঠিকানা বদলেছে কি না দেখুন। moved ব্লক বা state mv দিয়ে ঠিকানা মেলান; না পারলে পুরোনো সংস্করণে ফিরে যান।

৯. refresh-only plan-এ কী বদলেছে দেখুন। ইচ্ছাকৃত বদল হলে কনফিগারেশনে আনুন; না হলে apply দিয়ে drift ঠিক করুন। ভবিষ্যতে কনসোলে হাতে বদল নিষিদ্ধ করুন।

১০. তৎক্ষণাৎ apply থামান ও ভুল instance চিহ্নিত করুন। সঠিক সংখ্যায় ফিরিয়ে destroy করুন। count-এর variable-এ validation যোগ করুন ও CI-তে plan-এর সংখ্যা পর্যালোচনা বাধ্যতামূলক করুন।

কেস স্টাডির আলোচনা (১–৫)

১. মূল কারণ: local state-এর ওপর নির্ভরতা ও ব্যাকআপের অভাব। শিক্ষা: remote backend, versioning ও নিয়মিত ব্যাকআপ বাধ্যতামূলক; state-ই সত্যের উৎস, তাই তার সুরক্ষা সবচেয়ে আগে।

২. মূল কারণ: locking ছাড়া দলগত apply। শিক্ষা: local backend-এ দলগত কাজ নয়; remote backend ও locking ছাড়া একই state-এ দুজন কাজ করবে না।

৩. মূল কারণ: secret কোডে লেখা ও পর্যালোচনায় ধরা না পড়া। শিক্ষা: secret manager ব্যবহার, pre-commit secret scan, ও ফাঁস হলে তৎক্ষণাৎ key বাতিল করা — শুধু মুছে দিলেই হয় না।

৪. মূল কারণ: workspace যাচাই না করা, prevent_destroy না থাকা ও অনুমোদন প্রক্রিয়ার অভাব। শিক্ষা: destroy-এ অনুমোদন, workspace যাচাই, prevent_destroy ও ব্যাকআপ — চার স্তরের সুরক্ষা।

৫. মূল কারণ: module upgrade-এর আগে changelog ও plan পর্যালোচনা না করা। শিক্ষা: version pin, test পরিবেশে যাচাই, plan-এ destroy দেখলে থামা, ও moved ব্লকের ব্যবহার।

উত্তর-নির্দেশিকা (দ্বিতীয় অংশ) • Answer Guide Part 2

উত্তর-নির্দেশিকা — দ্বিতীয় অংশ

উত্তর-নির্দেশিকা — দ্বিতীয় অংশ

এই অংশে বহুনির্বাচনি প্রশ্ন ৭০–১৩৮, সংক্ষিপ্ত প্রশ্ন ১৬–৩০, পরিস্থিতি ১১–২০, কেস স্টাডি ৬–১০ ও চূড়ান্ত পরীক্ষার মূল্যায়ন নির্দেশনা আছে।

বহুনির্বাচনি প্রশ্নের উত্তর (৭০–১৩৮)

৭০. সঠিক উত্তর: খ) ইন্টারনেটের যে কেউ SSH চেষ্টা করতে পারে [Beginner]
0.0.0.0/0 মানে সবার জন্য খোলা; SSH শুধু নির্দিষ্ট IP-তে সীমিত রাখা উচিত।

৭১. সঠিক উত্তর: খ) অননুমোদিত ব্যক্তি ডেটা পড়তে বা বদলাতে পারে [Beginner]
Public bucket ডেটা ফাঁসের বড় কারণ; প্রয়োজনে ছাড়া বন্ধ রাখতে হয়।

৭২. সঠিক উত্তর: খ) রিসোর্স সংগঠিত করার ধারক (container) [Beginner]
Resource group Azure রিসোর্স গুছিয়ে রাখে ও একসাথে পরিচালনা সহজ করে।

৭৩. সঠিক উত্তর: খ) hashicorp/google [Beginner]
GCP রিসোর্সের জন্য hashicorp/google provider ব্যবহৃত হয়।

৭৪. সঠিক উত্তর: খ) IBM-Cloud/ibm [Intermediate]
IBM Cloud provider IBM-Cloud/ibm নামে পরিচিত।

৭৫. সঠিক উত্তর: খ) বর্তমান state বা plan ফাইলের মানব-পাঠযোগ্য বিবরণ [Beginner]
show state বা সংরক্ষিত plan-এর বিস্তারিত দেখায়।

৭৬. সঠিক উত্তর: খ) ঘোষিত output-এর মান দেখায় [Beginner]
output কমান্ড ঘোষিত output মানগুলো দেখায়।

৭৭. সঠিক উত্তর: খ) state-এর সব রিসোর্সের ঠিকানা [Intermediate]
state list state-এ থাকা রিসোর্স ঠিকানাগুলো তালিকা করে।

৭৮. সঠিক উত্তর: খ) state-এ রিসোর্সের ঠিকানা বদলায় (যেমন module-এ সরানো) [Advanced]
state mv refactoring-এর সময় রিসোর্সের state ঠিকানা বদলায়, বাস্তব রিসোর্স না মুছে।

৭৯. সঠিক উত্তর: খ) state থেকে রিসোর্স সরায়, বাস্তব রিসোর্স থেকে যায় [Advanced]
state rm শুধু state থেকে সরায়; বাস্তব রিসোর্স ক্লাউডে থেকে যায় ও পরে import করতে হয়।

৮০. সঠিক উত্তর: খ) রিসোর্স dependency-এর গ্রাফ (DOT ফরম্যাট) [Intermediate]
graph dependency কাঠামো দেখায়; জটিল নির্ভরতা বুঝতে কাজে লাগে।

৮১. সঠিক উত্তর: খ) কনফিগারেশনে ব্যবহৃত provider ও তাদের সংস্করণ [Beginner]
providers কমান্ড কোন provider ও সংস্করণ দরকার তা দেখায়।

৮২. সঠিক উত্তর: খ) Terraform ও provider-এর সংস্করণ [Beginner]
version Terraform CLI ও provider সংস্করণ দেখায়।

৮৩. সঠিক উত্তর: খ) changelog পড়ে non-production-এ plan যাচাই করা [Intermediate]
Breaking change production-এ যাওয়ার আগে changelog ও test environment-এ যাচাই করা উচিত।

৮৪. সঠিক উত্তর: খ) কোন provider, কোন source ও কোন সংস্করণ অনুমোদিত [Intermediate]
required_providers provider-এর source ও সংস্করণের সীমা নির্ধারণ করে।

৮৫. সঠিক উত্তর: খ) Terraform CLI-এর ন্যূনতম/অনুমোদিত সংস্করণ নির্ধারণ করে [Beginner]
required_version ভুল CLI সংস্করণে কাজ আটকায়।

৮৬. সঠিক উত্তর: খ) terraform init -migrate-state দিয়ে state স্থানান্তর করা [Advanced]
Backend বদলে init -migrate-state state নতুন backend-এ নিয়ে যায়।

৮৭. সঠিক উত্তর: খ) provider ও module-কে অনুমোদিত নতুন সংস্করণে আপডেট করে [Intermediate]
-upgrade lock ফাইলের সীমার মধ্যে নতুন সংস্করণ নেয়।

৮৮. সঠিক উত্তর: খ) পরের instance-গুলোর index সরে গিয়ে সেগুলো replace হতে পারে [Advanced]
count index-ভিত্তিক; মাঝেরটি মুছলে পরেরগুলোর index বদলে replace হতে পারে — for_each এ ক্ষেত্রে নিরাপদ।

৮৯. সঠিক উত্তর: খ) set-এর প্রতিটি মান [Intermediate]
set-এ key ও value একই মান হয়।

৯০. সঠিক উত্তর: খ) সাধারণত না — root module provider দেয়, child module তা উত্তরাধিকার পায় [Advanced]
Child module-এ provider হার্ডকোড করলে পুনর্ব্যবহার কমে; root থেকে provider পাস করা ভালো অভ্যাস।

৯১. সঠিক উত্তর: খ) ভুল মান (যেমন খালি নাম) আগেই ত্রুটি দিয়ে আটকায় [Intermediate]
validation ব্লক শর্ত না মিললে স্পষ্ট ত্রুটি দেয়; ভুল মান production-এ যাওয়া আটকায়।

৯২. সঠিক উত্তর: খ) .tftest.hcl ফাইলে লেখা assertion দিয়ে কনফিগারেশন যাচাই করে [Advanced]
terraform test mock provider দিয়ে plan/assert চালিয়ে যুক্তি যাচাই করে।

৯৩. সঠিক উত্তর: খ) Go-ভিত্তিক Terraform টেস্ট ফ্রেমওয়ার্ক যা আসল অবকাঠামো তৈরি করে যাচাই করে [Advanced]
Terratest Go দিয়ে লেখা টেস্টে আসল রিসোর্স তৈরি করে যাচাই করে, তারপর মুছে দেয়।

৯৪. সঠিক উত্তর: খ) plan থেকে আনুমানিক মাসিক খরচ দেখায় [Intermediate]
Infracost plan বিশ্লেষণ করে খরচের অনুমান PR-এ দেখায়।

৯৫. সঠিক উত্তর: খ) অপ্রত্যাশিতভাবে 100টি রিসোর্স তৈরি হয়ে বিল বিস্ফোরণ হতে পারে [Intermediate]
count ভুল হলে বড় সংখ্যক রিসোর্স তৈরি হয়; plan-এ সংখ্যা যাচাই করা জরুরি।

৯৬. সঠিক উত্তর: খ) apply থামিয়ে কারণ খোঁজা — নাম, key বা কনফিগারেশন বদলেছে কি না [Intermediate]
অপ্রত্যাশিত destroy প্রায়ই নাম/key বদল বা module সরানোর লক্ষণ; apply করার আগে কারণ নিশ্চিত করতে হয়।

৯৭. সঠিক উত্তর: খ) আর কোনো apply চলছে কি না; চললে অপেক্ষা করা [Intermediate]
চলমান apply থাকলে force-unlock state নষ্ট করতে পারে; আগে নিশ্চিত হতে হয় কেউ apply করছে না।

৯৮. সঠিক উত্তর: খ) যখন নিশ্চিত যে কোনো apply চলছে না, কিন্তু lock আটকে আছে [Advanced]
force-unlock শুধু আটকে থাকা lock খোলে; চলমান apply-এর lock খুললে state corrupt হতে পারে।

৯৯. সঠিক উত্তর: খ) encrypted remote backend + কঠোর IAM + state-এর access সীমিত করে [Intermediate]
state-এ secret থাকতে পারে; encryption ও access নিয়ন্ত্রণই প্রধান সুরক্ষা।

১০০. সঠিক উত্তর: খ) state হারিয়ে যায়; ব্যাকআপ/versioning না থাকলে পুনরুদ্ধার কঠিন হয় [Advanced]
state-ই সত্যের উৎস; bucket-এর versioning ও ব্যাকআপ না থাকলে পুনরুদ্ধার প্রায় অসম্ভব।

১০১. সঠিক উত্তর: খ) না — state-এ secret থাকতে পারে ও দলগত lock হয় না [Beginner]
state Git-এ রাখা নিরাপত্তা ও concurrency ঝুঁকি তৈরি করে; remote backend ব্যবহার করা উচিত।

১০২. সঠিক উত্তর: খ) .terraform/, *.tfstate, *.tfplan, crash.log [Beginner]
state, plan ও .terraform ডিরেক্টরি Git-এ যাওয়া উচিত নয়।

১০৩. সঠিক উত্তর: খ) বর্তমান workspace dev-এ বদলায় [Beginner]
select বর্তমান workspace বদলায়; পরের plan/apply সেই state-এ হবে।

১০৪. সঠিক উত্তর: খ) আলাদা ডিরেক্টরি/আলাদা backend key বা tfvars ফাইল ব্যবহার করা [Intermediate]
আলাদা tfvars/backend key দিয়ে environment আলাদা করা যায়; বড় দলে এটাই বেশি ব্যবহৃত হয়।

১০৫. সঠিক উত্তর: খ) variable-এর মান ফাইলে দিয়ে পরিবেশভেদে ব্যবহার করা [Beginner]
terraform.tfvars বা -var-file দিয়ে variable মান দেওয়া যায়।

১০৬. সঠিক উত্তর: খ) TF_VAR_<নাম> দিয়ে variable-এর মান দেয় [Intermediate]
TF_VAR_ উপসর্গ দিয়ে environment variable থেকে variable মান দেওয়া যায়।

১০৭. সঠিক উত্তর: খ) শুধু নির্দিষ্ট রিসোর্স ও তার dependency নিয়ে plan করে [Advanced]
-target শুধু নির্দিষ্ট রিসোর্সে সীমিত plan/apply করে; নিয়মিত ব্যবহারে drift লুকাতে পারে, তাই সাবধানে ব্যবহার করা উচিত।

১০৮. সঠিক উত্তর: খ) অপ্রয়োজনীয় ক্রম তৈরি হয়ে parallel কাজ কমে ও graph জটিল হয় [Advanced]
অপ্রয়োজনীয় depends_on concurrency কমায়; implicit dependency যথেষ্ট হলে তা-ই ভালো।

১০৯. সঠিক উত্তর: খ) যখন এমন argument বদলায় যা ForceNew — যেমন নাম বা AMI [Intermediate]
কিছু argument (যেমন instance-এর AMI) বদলালে রিসোর্স ধ্বংস করে নতুন তৈরি করতে হয়।

১১০. সঠিক উত্তর: খ) plan-এর সময় (বেশিরভাগ ক্ষেত্রে) [Intermediate]
Data source সাধারণত plan-এ পড়া হয়; তবে dependency থাকলে apply-এর সময় পর্যন্ত পিছিয়ে যেতে পারে।

১১১. সঠিক উত্তর: খ) এর মান state-এ থাকে ও আউটপুটে গোপন রাখা উচিত [Intermediate]
random_password-এর মান state-এ থাকে; output-এ sensitive করা ও state সুরক্ষিত রাখা জরুরি।

১১২. সঠিক উত্তর: খ) provisioner বা ট্রিগার-ভিত্তিক কাজ চালাতে (বর্তমানে terraform_data দিয়ে প্রতিস্থাপিত) [Advanced]
null_resource কোনো ক্লাউড রিসোর্স তৈরি করে না; provisioner চালানোর জন্য ব্যবহৃত হতো।

১১৩. সঠিক উত্তর: খ) শেষ উপায় হিসেবে ব্যবহার করা; তার আগে provider/cloud-init ব্যবহার করা [Advanced]
Provisioner ভঙ্গুর; cloud-init বা provider-native উপায়কে অগ্রাধিকার দেওয়া উচিত।

১১৪. সঠিক উত্তর: খ) Terraform চালানো মেশিনে কমান্ড চালায় [Advanced]
local-exec Terraform চলা মেশিনে কমান্ড চালায়; CI-নির্ভরতা তৈরি করে।

১১৫. সঠিক উত্তর: খ) তৈরি রিসোর্সে SSH/WinRM দিয়ে কমান্ড চালায় [Advanced]
remote-exec তৈরি সার্ভারে সংযোগ করে কমান্ড চালায়।

১১৬. সঠিক উত্তর: খ) Terraform Cloud/Enterprise private registry বা Git source দিয়ে [Advanced]
Private module Terraform Cloud registry বা Git source দিয়ে ব্যবহার করা যায়।

১১৭. সঠিক উত্তর: খ) URL-এ ?ref=<tag/branch> দিয়ে [Advanced]
Git module-এ ?ref=v1.2.0 দিয়ে নির্দিষ্ট tag/branch বেছে নেওয়া যায়।

১১৮. সঠিক উত্তর: খ) backend কনফিগার না করে শুধু module/provider প্রস্তুত করতে [Advanced]
-backend=false backend ছাড়া init করে; CI-তে validation-এর সময় কাজে লাগে।

১১৯. সঠিক উত্তর: খ) বাস্তব অবস্থা কনফিগারেশনের সাথে মিলে গেছে [Beginner]
"No changes" মানে drift নেই ও কনফিগারেশন ইতিমধ্যে প্রয়োগ করা আছে।

১২০. সঠিক উত্তর: খ) ত্রুটি ঠিক করে আবার plan/apply করা; state-এ আংশিক অবস্থা থাকে [Intermediate]
ব্যর্থ apply-তে state আংশিক আপডেট থাকে; ঠিক করে পুনরায় apply করলেই Terraform বাকিটা মেলায়।

১২১. সঠিক উত্তর: খ) lock নিতে ব্যর্থ হয়ে init/apply ত্রুটি দেয় [Intermediate]
ভুল lock টেবিলে Terraform lock নিতে পারে না, তাই backend init ব্যর্থ হয়।

১২২. সঠিক উত্তর: খ) Blob lease দিয়ে [Intermediate]
azurerm backend blob lease দিয়ে lock ধারণ করে।

১২৩. সঠিক উত্তর: খ) GCS object-এর মাধ্যমে (native locking) [Advanced]
gcs backend GCS-এর মাধ্যমে lock পরিচালনা করে।

১২৪. সঠিক উত্তর: খ) শুধু একটি রিসোর্স মুছে dependency ভেঙে যেতে পারে [Advanced]
-target দিয়ে আংশিক destroy করলে নির্ভরশীল রিসোর্স ভেঙে যেতে পারে।

১২৫. সঠিক উত্তর: খ) Terraform মান থেকে type অনুমান করে (dynamic) [Beginner]
type না দিলে type dynamic হয়; তবে স্পষ্ট type দেওয়া নিরাপদ।

১২৬. সঠিক উত্তর: খ) CIDR ব্লক থেকে সাবনেট বের করে [Advanced]
cidrsubnet একটি বড় CIDR থেকে নির্দিষ্ট সাবনেট CIDR গণনা করে।

১২৭. সঠিক উত্তর: খ) string বড় করতে [Intermediate]
toset list-কে set করে, যা for_each-এর key হিসেবে ব্যবহার করা যায়।

১২৮. সঠিক উত্তর: খ) map-এ key থাকলে তার মান, না থাকলে default দেয় [Intermediate]
lookup নিরাপদে map থেকে মান নেয়; key না থাকলে default ফেরত দেয়।

১২৯. সঠিক উত্তর: খ) লুপ চালানো [Advanced]
try প্রথম expression ব্যর্থ হলে পরেরটি চেষ্টা করে; optional attribute-এ কাজে লাগে।

১৩০. সঠিক উত্তর: খ) প্রথম non-empty/non-null মান ফেরত দেয় [Intermediate]
coalesce একাধিক মানের মধ্যে প্রথম খালি নয় এমন মান দেয়।

১৩১. সঠিক উত্তর: খ) Terraform মানকে JSON string-এ রূপান্তর করে [Intermediate]
jsonencode map/object-কে JSON string করে; IAM policy-তে কাজে লাগে।

১৩২. সঠিক উত্তর: খ) টেমপ্লেট ফাইল পড়ে variable বসিয়ে string তৈরি করে [Intermediate]
templatefile user-data বা policy টেমপ্লেটে মান বসাতে ব্যবহৃত হয়।

১৩৩. সঠিক উত্তর: খ) ঠিকানা বদলালেও (যেমন module-এ সরালে) state ধরে রেখে recreation আটকায় [Advanced]
moved ব্লক পুরোনো ঠিকানা থেকে নতুন ঠিকানায় state স্থানান্তরের ঘোষণা দেয়।

১৩৪. সঠিক উত্তর: খ) কনফিগারেশন থেকে সরানো রিসোর্স state থেকেও সরায়, বাস্তব রিসোর্স না মুছে [Advanced]
removed ব্লক lifecycle destroy ছাড়া state থেকে রিসোর্স সরায়।

১৩৫. সঠিক উত্তর: খ) apply-এর পর assertion দিয়ে অবস্থা যাচাই করে [Advanced]
check ব্লক apply-এর পর শর্ত যাচাই করে; ব্যর্থ হলে সতর্কতা দেয়।

১৩৬. সঠিক উত্তর: খ) কনফিগারেশনে লিখে import করা যায়, ফলে পর্যালোচনাযোগ্য হয় [Advanced]
import ব্লক import-কে কোডে লেখে, Git পর্যালোচনায় আনা যায়।

১৩৭. সঠিক উত্তর: খ) state-এ সংরক্ষণ না করা অস্থায়ী সংবেদনশীল মান [Advanced]
ephemeral মান state-এ লেখা হয় না; secret সাময়িকভাবে ব্যবহারে কাজে লাগে।

১৩৮. সঠিক উত্তর: খ) শুধু লেখা যায়, state-এ সংরক্ষণ হয় না এমন argument [Advanced]
write-only argument (যেমন পাসওয়ার্ড) state-এ সংরক্ষণ হয় না।

সংক্ষিপ্ত প্রশ্নের উত্তর (১৬–৩০)

১৬. Policy as Code মানে নিয়ম কোডে লেখা ও স্বয়ংক্রিয় যাচাই। উদাহরণ: OPA-তে ‘সব S3 bucket-এ encryption থাকতে হবে’ — plan যাচাইয়ে নিয়ম না মানলে apply আটকে যায়। Checkov-ও একইভাবে কনফিগারেশন যাচাই করে।

১৭. ধাপ: (১) fmt দিয়ে বিন্যাস ঠিক করা; (২) validate দিয়ে syntax যাচাই; (৩) PR-এ plan চালিয়ে পর্যালোচনা; (৪) merge-এর পর অনুমোদিত apply। plan-এর আউটপুট PR-এ দেখানো হয়, যাতে destroy আগেই ধরা পড়ে।

১৮. terraform test built-in ফ্রেমওয়ার্ক — .tftest.hcl-এ assertion লিখে mock provider দিয়ে যুক্তি যাচাই করে, আসল resource তৈরি করে না। Terratest Go দিয়ে লেখা হয় ও আসল অবকাঠামো তৈরি করে যাচাই করে, তারপর মুছে দেয় — ধীর কিন্তু বাস্তব।

১৯. যাচাই: Git diff-এ নাম/argument বদলেছে কি না; module সংস্করণ বদলেছে কি না; for_each/count key বদলেছে কি না; state-এ resource আছে কি না; workspace ঠিক আছে কি না। কারণ না পাওয়া পর্যন্ত apply নয়।

২০. State-এ আংশিক অবস্থা থাকে — কিছু resource তৈরি হয়েছে, কিছু হয়নি। ত্রুটি ঠিক করে আবার plan/apply করুন; Terraform বাকিটা মেলাবে। state মুছে দেবেন না — তাতে ডুপ্লিকেট resource তৈরি হবে।

২১. প্রথমে changelog পড়ুন। ঠিকানা বদলে থাকলে moved ব্লক লিখুন বা terraform state mv দিয়ে পুরোনো ঠিকানা নতুন ঠিকানায় নিন। যাচাই না করে apply করবেন না; প্রয়োজনে পুরোনো সংস্করণে ফিরুন।

২২. ধাপ: (১) নতুন backend কনফিগার করুন; (২) terraform init -migrate-state দিয়ে state স্থানান্তর করুন; (৩) plan দিয়ে যাচাই করুন; (৪) পুরোনো backend-এর ব্যাকআপ রাখুন। মাঝপথে কেউ apply করছে না নিশ্চিত করুন।

২৩. Plan-এ resource-এর সংখ্যা, instance-এর আকার ও নতুন ব্যয়বহুল সেবা আছে কি না দেখুন। Infracost plan থেকে আনুমানিক মাসিক খরচ দেখায়; বড় পরিবর্তনে budget alert ও অনুমোদন নিন।

২৪. Provisioner ভঙ্গুর — নেটওয়ার্ক বা SSH ব্যর্থ হলে apply ব্যর্থ হয়, আর provisioner-এর কাজ state-এ ধরা পড়ে না। বিকল্প: cloud-init/user-data, provider-native কনফিগারেশন, বা Packer দিয়ে image তৈরি করা।

২৫. state mv state-এ resource-এর ঠিকানা বদলায়, বাস্তব resource অক্ষত থাকে — refactoring-এ ব্যবহৃত হয়। state rm state থেকে resource সরায়, বাস্তব resource ক্লাউডে থেকে যায় — Terraform আর তা পরিচালনা করে না।

২৬. Checklist: (১) সঠিক workspace ও backend যাচাই; (২) plan পড়ে destroy/replace আছে কি না দেখা; (৩) সংখ্যা ও আকার যাচাই; (৪) ব্যাকআপ আছে কি না; (৫) অনুমোদন নেওয়া হয়েছে কি না; (৬) apply-এর পর যাচাইয়ের পরিকল্পনা।

পরিস্থিতির সমাধান (১১–২০)

১১. ত্রুটি ঠিক করে আবার plan/apply করুন। state-এ আংশিক অবস্থা থাকে — state মুছবেন না। নেটওয়ার্ক ঠিক আছে কি না ও backend-এ lock আটকে আছে কি না দেখুন।

১২. lock timeout বাড়ানোর সেটিং আছে কি না দেখুন; apply ছোট ভাগে করুন; backend-এর স্বাস্থ্য যাচাই করুন। দীর্ঘ apply-এ timeout স্বাভাবিক হতে পারে — সেটিং বদলে সমাধান হয়।

১৩. তৎক্ষণাৎ apply থামান ও কী বদলেছে যাচাই করুন। ভুল resource ফিরিয়ে আনুন। ভবিষ্যতে prompt-এ workspace নাম দেখানো ও destroy-এ অনুমোদন বাধ্যতামূলক করুন।

১৪. তৎক্ষণাৎ key বাতিল (revoke) করুন — শুধু মুছে দিলেই হয় না, কারণ history-তে key থাকে। তারপর history পরিষ্কার করুন ও secret manager-এ সরান।

১৫. তৎক্ষণাৎ destroy থামান। state ব্যাকআপ থাকলে পুনরুদ্ধারের চেষ্টা করুন। ভবিষ্যতে গুরুত্বপূর্ণ resource-এ prevent_destroy ও destroy-এ অনুমোদন যোগ করুন।

১৬. changelog পড়ে নতুন argument অনুযায়ী কনফিগারেশন ঠিক করুন। test পরিবেশে যাচাই করুন। ভবিষ্যতে provider সংস্করণ pin করুন ও major upgrade আলাদাভাবে পরীক্ষা করুন।

১৭. terraform fmt চালিয়ে বিন্যাস ঠিক করুন; validate-এর ত্রুটির বার্তা পড়ে ভুল reference ঠিক করুন। CI-তে fmt -check বাধ্যতামূলক রাখুন।

১৮. data source-এর filter সুনির্দিষ্ট করুন (যেমন newest AMI-এর filter)। dependency ঠিক আছে কি না দেখুন; প্রয়োজনে depends_on দিয়ে ক্রম ঠিক করুন।

১৯. bucket-এর versioning থেকে ফিরিয়ে আনার চেষ্টা করুন; না পারলে local ব্যাকআপ state দিয়ে পুনর্নির্মাণ করুন। ভবিষ্যতে backend bucket-এ versioning, MFA delete ও deletion protection চালু করুন।

২০. output-এ sensitive = true যোগ করুন। state-এর access সীমিত করুন ও সংশ্লিষ্ট পাসওয়ার্ড rotate করুন — আউটপুটে দেখা গেলেও state-এ মান থাকে।

কেস স্টাডির আলোচনা (৬–১০)

৬. মূল কারণ: drift শনাক্ত না করা ও কনসোলে হাতে বদল। শিক্ষা: refresh-only plan নিয়মিত চালানো, হাতে বদল নিষিদ্ধ করা, ও apply-এর আগে drift যাচাই করা।

৭. মূল কারণ: backend bucket-এ সুরক্ষার অভাব। শিক্ষা: backend bucket-এ versioning, MFA delete, deletion protection ও নিয়মিত ব্যাকআপ — state হারানো মানে সব কাজ থেমে যাওয়া।

৮. মূল কারণ: দীর্ঘ apply ও lock সেটিং-এর অভাব। শিক্ষা: apply ছোট ভাগে করা, lock timeout যাচাই করা, ও আংশিক apply-এর পর state যাচাই করে আবার apply করা।

৯. মূল কারণ: changelog না পড়ে major upgrade করা। শিক্ষা: version pin, test পরিবেশে যাচাই, ও major upgrade-কে আলাদা পরিবর্তন হিসেবে পর্যালোচনা করা।

১০. মূল কারণ: plan-এ সংখ্যা যাচাই না করা ও cost check-এর অভাব। শিক্ষা: plan-এ সংখ্যা দেখা, Infracost, budget alert, ও variable-এ validation যোগ করা।

চূড়ান্ত পরীক্ষার মূল্যায়ন নির্দেশনা

প্রতিটি কাজে নম্বর দেওয়ার সময় তিনটি বিষয় দেখুন — (১) কাজটি সম্পূর্ণ হয়েছে কি না, (২) plan পড়ে ঝুঁকি বোঝা হয়েছে কি না, (৩) নিরাপত্তা ও পরিচ্ছন্নতা (destroy, secret, backend) মানা হয়েছে কি না। শুধু resource তৈরি হলেই পূর্ণ নম্বর নয়; ভুল workspace-এ apply, অপ্রত্যাশিত destroy উপেক্ষা করা বা secret কোডে লেখা থাকলে নম্বর কাটা যাবে। কাজ ১-এ module-এর variable, output ও README — তিনটিই থাকতে হবে। কাজ ৩-এ locking-এর প্রমাণ (যেমন দ্বিতীয় apply-এ lock বার্তা) দেখাতে হবে। কাজ ৪-এ import-এর পর ‘No changes’ না দেখাতে পারলে পূর্ণ নম্বর নয়। কাজ ৫-এ destroy/replace ও খরচের প্রভাব উল্লেখ না থাকলে নম্বর কাটা যাবে।

> **মূল্যায়কের জন্য:** পরীক্ষার্থীর plan আউটপুটগুলো সংরক্ষণ করে রাখুন; ভবিষ্যতে একই পরীক্ষার্থীর অগ্রগতি মাপতে এগুলোই প্রমাণ।